MATH 4432 – Assignment 1

Coding parts: Chapter 2 exercises 8 and 10, Chapter 3 exercises 8 and 9. The conceptual exercises (Ch. 2: 1, 3; Ch. 3: 1, 5) are in the separate PDF.

In [1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import scipy.stats as stats
from scipy.stats import t
import statsmodels.api as sm
from statsmodels.formula.api import ols
from statsmodels.nonparametric.smoothers_lowess import lowess
from statsmodels.stats.diagnostic import het_breuschpagan
from ISLP import load_data

%matplotlib inline
```

Helper functions used in several exercises.

In [2]:
```
def corr_heatmap(df, title, figsize=(11, 9)):
    """Annotated correlation heatmap of the numeric columns of df."""
    corr = df.corr(numeric_only=True)
    cols = corr.columns
    fig, ax = plt.subplots(figsize=figsize)
    im = ax.imshow(corr, cmap='RdBu_r', vmin=-1, vmax=1)
    ax.set_xticks(range(len(cols)))
    ax.set_yticks(range(len(cols)))
    ax.set_xticklabels(cols, rotation=90)
    ax.set_yticklabels(cols)
    for i in range(len(cols)):
        for j in range(len(cols)):
            ax.text(j, i, f'{corr.iloc[i, j]:.2f}', ha='center', va='center'
                    color='white' if abs(corr.iloc[i, j]) > 0.6 else 'black'
    fig.colorbar(im, ax=ax, shrink=0.8, label='Correlation')
    ax.set_title(title)
    fig.tight_layout()
    plt.show()
    return corr


def strongest_pairs(corr, n_pos=8, n_neg=5):
    """Strongest pairwise correlations, excluding the diagonal."""
    pairs = corr.where(np.triu(np.ones(corr.shape, dtype=bool), k=1)).stack(
    print("Strongest positive correlations:")
    display(pairs.head(n_pos).round(3))
    print("Strongest negative correlations:")
    display(pairs.tail(n_neg).round(3))
```

Chapter 2

Exercises: 1, 3, 8, 10

Exercise 8 – College

(a)–(b) Read the data and use the college name as index.

In [3]:
```
college = pd.read_csv('College.csv')
# The first column holds the college names; use it as the row index
# (equivalent to pd.read_csv('College.csv', index_col=0))
college = college.rename({'Unnamed: 0': 'College'}, axis=1).set_index('Colle
college
```

Out[3]:

College	Private	Apps	Accept	Enroll	Top10perc	Top25perc	F.Undergrad	P.U
Abilene Christian University	Yes	1660	1232	721	23	52	2885	
Adelphi University	Yes	2186	1924	512	16	29	2683	
Adrian College	Yes	1428	1097	336	22	50	1036	
Agnes Scott College	Yes	417	349	137	60	89	510	
Alaska Pacific University	Yes	193	146	55	16	44	249	
...	...	...	...	...	...	...	...	
Worcester State College	No	2197	1515	543	4	26	3089	
Xavier University	Yes	1959	1805	695	24	47	2849	
Xavier University of Louisiana	Yes	2097	1915	695	34	61	2793	
Yale University	Yes	10705	2453	1317	95	99	5217	
York College of Pennsylvania	Yes	2989	1855	691	28	63	2988	

777 rows × 18 columns

(c) Numerical summary of the variables.

In [4]:
```
college.describe()
```

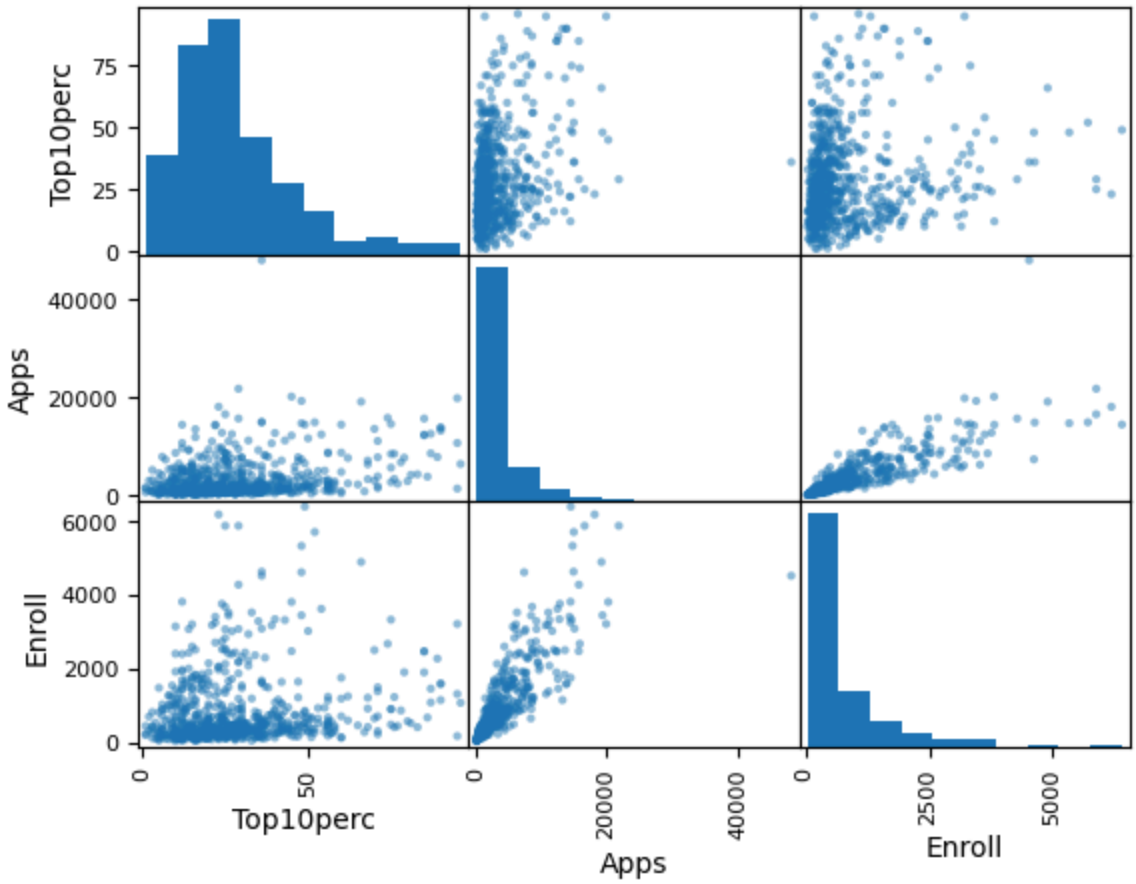
Out[4]:

	Apps	Accept	Enroll	Top10perc	Top25perc	F.Under
count	777.000000	777.000000	777.000000	777.000000	777.000000	777.000
mean	3001.638353	2018.804376	779.972973	27.558559	55.796654	3699.907
std	3870.201484	2451.113971	929.176190	17.640364	19.804778	4850.42
min	81.000000	72.000000	35.000000	1.000000	9.000000	139.000
25%	776.000000	604.000000	242.000000	15.000000	41.000000	992.000
50%	1558.000000	1110.000000	434.000000	23.000000	54.000000	1707.000
75%	3624.000000	2424.000000	902.000000	35.000000	69.000000	4005.000
max	48094.000000	26330.000000	6392.000000	96.000000	100.000000	31643.000

Comment (c). The data set has 777 colleges and 18 variables, none missing. Most count variables are strongly right-skewed: `Apps` has median 1 558 but mean 3 002 and maximum 48 094, and the same holds for `Accept`, `Enroll`, `F.Undergrad` and `P.Undergrad`, so a few very large universities pull the means far above the medians. `Outstate` ranges from 2 340 to 21 700 dollars with median 9 990. Two values are impossible: `Grad.Rate` has a maximum of 118 % and `PhD` a maximum of 103 %, which must be data entry errors. Median `S.F.Ratio` is 13.6 students per faculty member and median `Grad.Rate` is 65 %.

(d) Scatterplot matrix of `Top10perc`, `Apps`, `Enroll`.

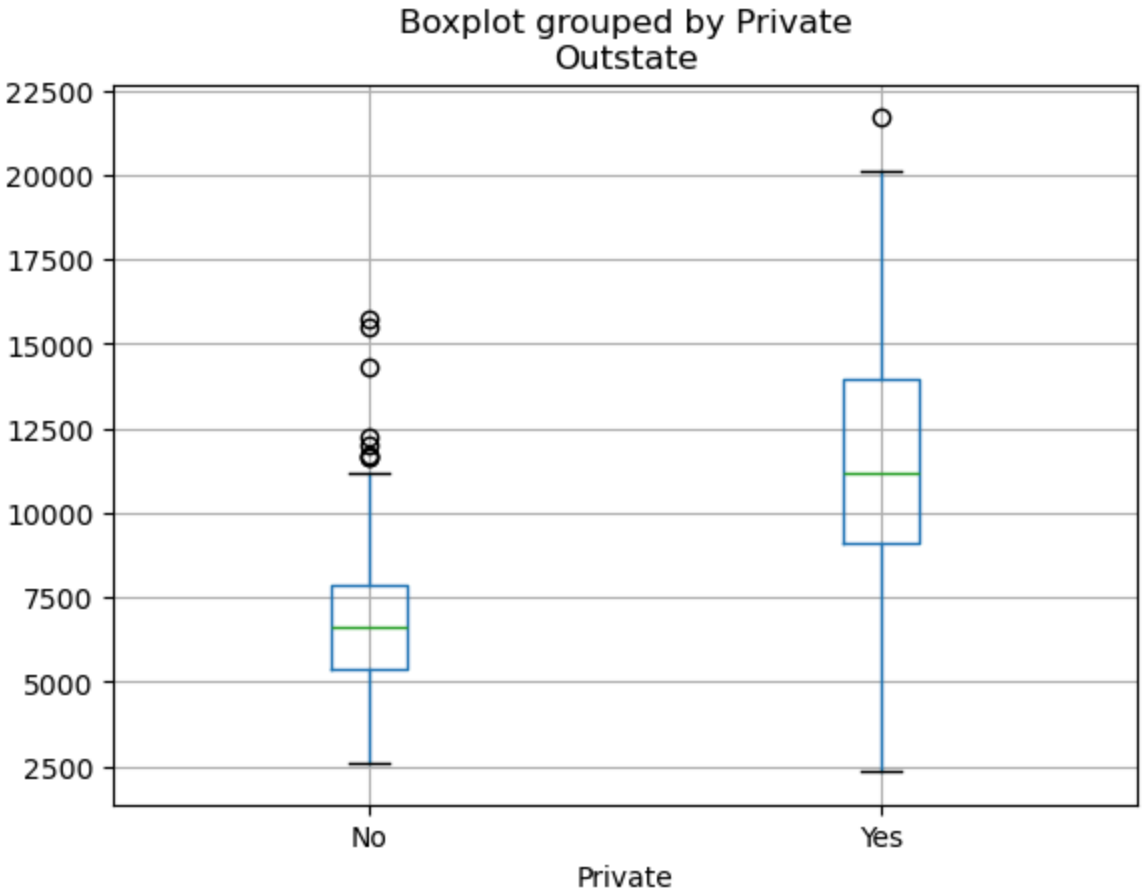
In [5]:
```
columns = ["Top10perc", "Apps", "Enroll"]
pd.plotting.scatter_matrix(college[columns])
plt.show()
```



Comment (d). `Apps` and `Enroll` are strongly positively related, as expected since enrolment is bounded by applications, and the relationship is roughly linear with a few large state universities in the upper right. `Top10perc` has a weaker, positive but noisy relationship with both: colleges with many top-10 % students receive more applications, but there are many small selective colleges with few applications. The diagonal histograms confirm the right skew seen in (c).

(e) `Outstate` versus `Private`.

In [6]:
```
college.boxplot(column='Outstate', by='Private')
plt.show()
```

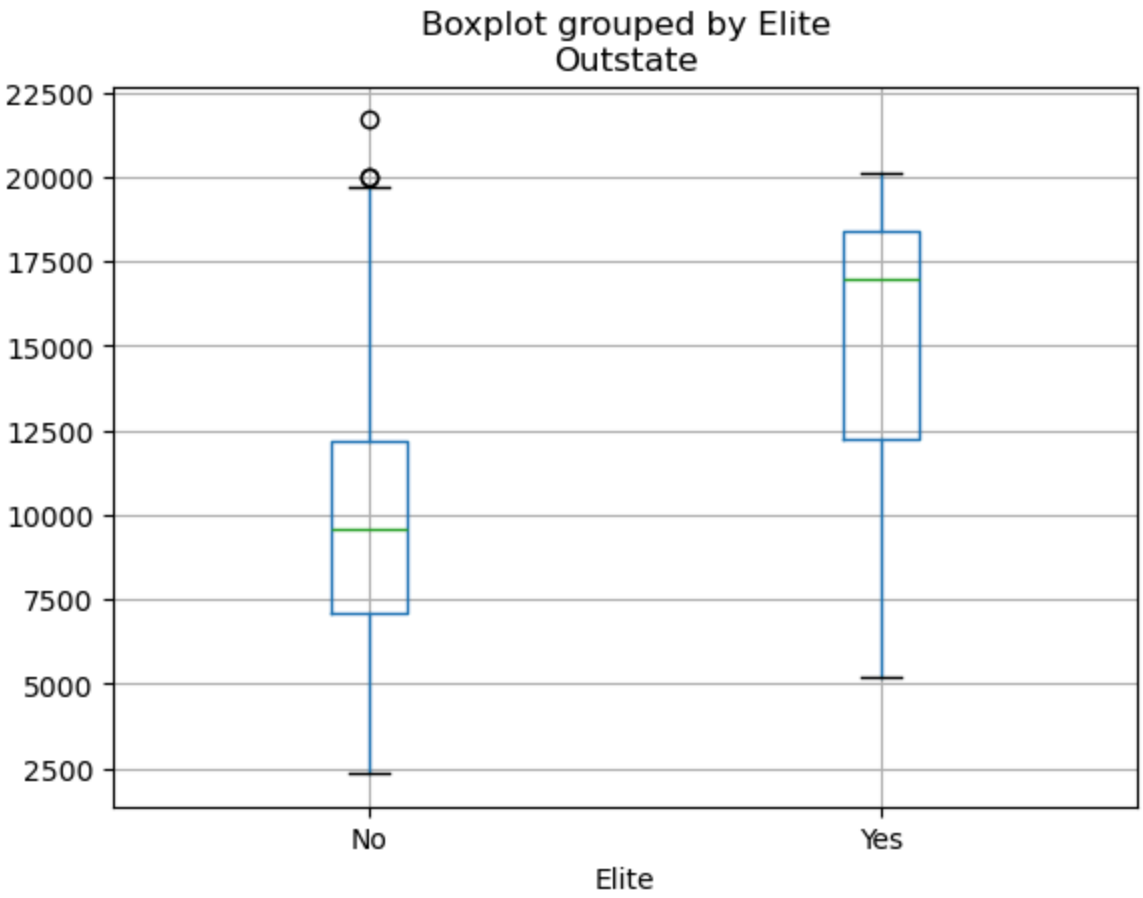


Comment (e). Private colleges charge clearly more out-of-state tuition: the median is roughly 11 000 dollars for private colleges against roughly 6 600 dollars for public ones, and the interquartile ranges barely overlap. Private colleges also show a much wider spread, from about 2 300 to 21 700 dollars, whereas public colleges are concentrated in a narrow band. Public colleges are largely state-funded, so out-of-state tuition does not need to cover the full cost.

(f) Elite universities: more than 50 % of new students from the top 10 % of their high school class.

In [7]:
```
# Top10perc is in percent (0–100), so the cut points are 50 and 100 rather t
college['Elite'] = pd.cut(college['Top10perc'], [0, 50, 100], labels=['No',
print(college['Elite'].value_counts())
college.boxplot(column='Outstate', by='Elite')
plt.show()
```

```
Elite
No     699
Yes     78
Name: count, dtype: int64
```



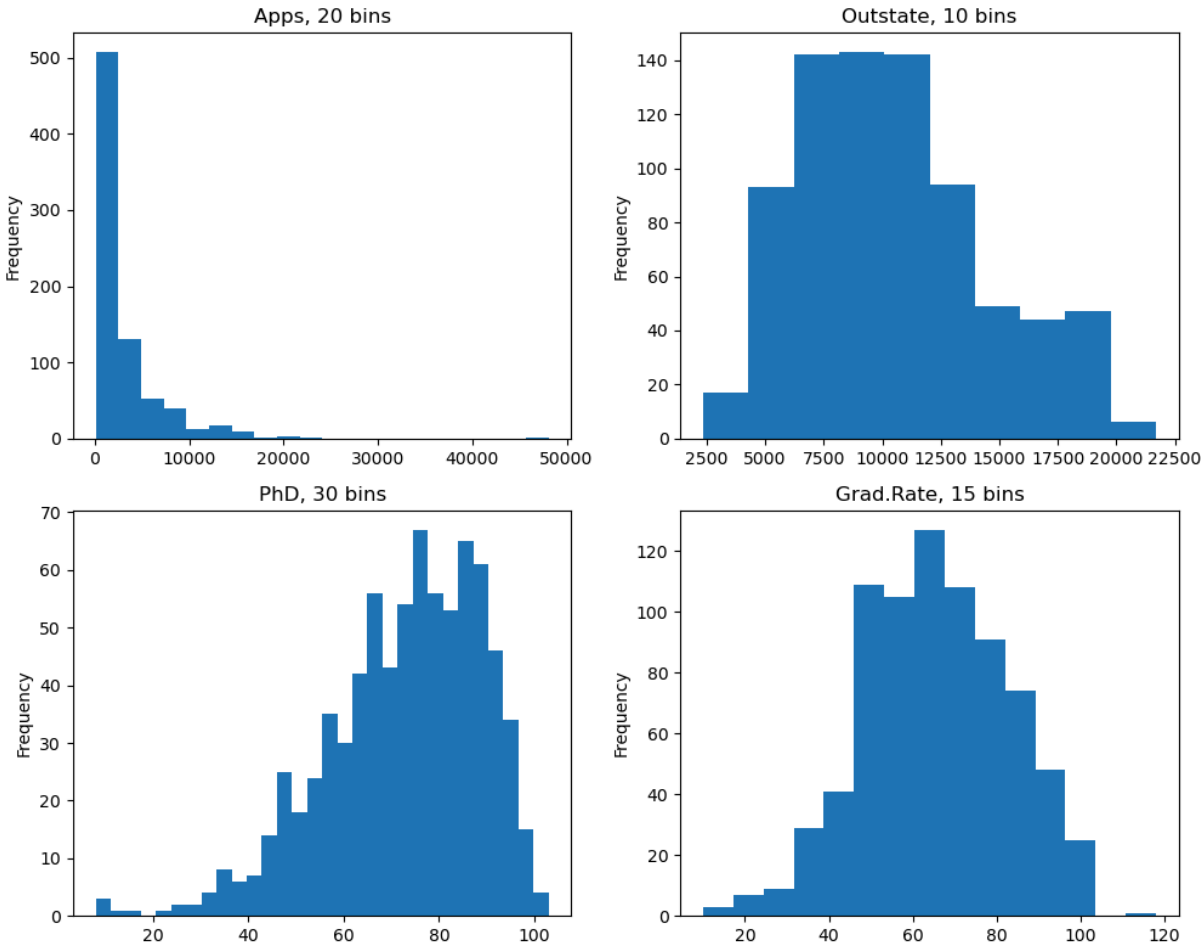
Comment (f). Note that `Top10perc` is measured in percent, so the cut points are 50 and 100 rather than the book's 0.5 and 1 (with the book's values only 3 colleges would be classified). With the correct cut points 78 of 777 colleges (10 %) are elite. Elite colleges have much higher out-of-state tuition, median about 15 000 dollars against about 10 000 dollars, and the lower quartile of the elite group lies above the median of the non-elite group.

(g) Histograms with different numbers of bins.

In [8]:
```
fig, axes = plt.subplots(2, 2, figsize=(10, 8))

college['Apps'].plot.hist(bins=20, ax=axes[0, 0], title='Apps, 20 bins')
college['Outstate'].plot.hist(bins=10, ax=axes[0, 1], title='Outstate, 10 bi
college['PhD'].plot.hist(bins=30, ax=axes[1, 0], title='PhD, 30 bins')
college['Grad.Rate'].plot.hist(bins=15, ax=axes[1, 1], title='Grad.Rate, 15

fig.tight_layout()
plt.show()
```



Comment (g). `Apps` is extremely right-skewed: most colleges receive fewer than 5 000 applications and a long tail reaches 48 000, so with 20 bins almost all mass sits in the first two. `Outstate` is roughly symmetric and bimodal-looking with 10 bins, reflecting the public/private split seen in (e). `PhD` is left-skewed with most colleges between 60 and 95 %. `Grad.Rate` is approximately symmetric around 65 % but the 15-bin histogram shows the impossible value above 100. Changing the number of bins mostly changes how much detail is visible; too few bins hide the bimodality of `Outstate`, too many make `Apps` look like a single spike.

(h) Further exploration.

In [9]:
```
# Elite vs non-elite, acceptance rate and correlations

# 1. Elite vs non-elite: mean of every variable
print("Mean per group (Elite):")
display(college.groupby('Elite', observed=True).mean(numeric_only=True).T.rc

# 2. Median and mean of a few key variables
key_cols = ['Outstate', 'Expend', 'Grad.Rate', 'S.F.Ratio', 'PhD', 'perc.alu
print("Median and mean of key variables:")
display(college.groupby('Elite', observed=True)[key_cols].agg(['median', 'me

# 3. Acceptance rate
college['AcceptRate'] = college['Accept'] / college['Apps']
```

```
print("Acceptance rate by Elite:")
display(college.groupby('Elite', observed=True)['AcceptRate'].describe().rou
print("Acceptance rate by Private:")
display(college.groupby('Private')['AcceptRate'].describe().round(3))
print("Number of colleges by Private x Elite:")
display(pd.crosstab(college['Private'], college['Elite']))

fig, axes = plt.subplots(1, 3, figsize=(15, 4.5))
college.boxplot(column='AcceptRate', by='Elite', ax=axes[0])
axes[0].set_title('Acceptance rate vs Elite')
college.boxplot(column='AcceptRate', by='Private', ax=axes[1])
axes[1].set_title('Acceptance rate vs Private')
axes[2].scatter(college['AcceptRate'], college['Grad.Rate'], alpha=0.5)
axes[2].set_xlabel('Acceptance rate')
axes[2].set_ylabel('Grad.Rate')
axes[2].set_title('Grad.Rate vs acceptance rate')
fig.suptitle('')
fig.tight_layout()
plt.show()

# 4. Correlation with Grad.Rate
corr_grad = college.corr(numeric_only=True)['Grad.Rate'].drop('Grad.Rate').s
print("Correlation with Grad.Rate:")
display(corr_grad.round(3))

fig, ax = plt.subplots(figsize=(8, 5))
corr_grad.plot.barh(ax=ax)
ax.set_xlabel('Correlation with Grad.Rate')
ax.axvline(0, color='black', linewidth=0.8)
fig.tight_layout()
plt.show()

# 5. Impossible values: Grad.Rate above 100 %
print("Colleges with Grad.Rate > 100:")
display(college.loc[college['Grad.Rate'] > 100, ['Private', 'Apps', 'Grad.Ra
```

```
Mean per group (Elite):
```

Elite	No	Yes
Apps	2669.2	5980.6
Accept	1925.8	2852.6
Enroll	748.6	1060.7
Top10perc	23.1	67.6
Top25perc	51.9	91.1
F.Undergrad	3601.4	4582.7
P.Undergrad	914.5	324.9
Outstate	9904.2	15248.6
Room.Board	4248.3	5336.8
Books	544.3	594.9
Personal	1357.7	1188.2
PhD	70.8	89.3
Terminal	78.1	94.1
S.F.Ratio	14.5	10.6
perc.alumni	21.5	34.0
Expend	8684.4	18404.9
Grad.Rate	63.5	83.4

Median and mean of key variables:

	Outstate		Expend		Grad.Rate		S.F.Ratio		PhD	
	median	mean	median	mean	median	mean	median	mean	median	mean
Elite										
No	9556.0	9904.2	8086.0	8684.4	64.0	63.5	13.9	14.5	73.0	70.8
Yes	16950.0	15248.6	15944.0	18404.9	84.5	83.4	10.4	10.6	91.0	89.3

Acceptance rate by Elite:

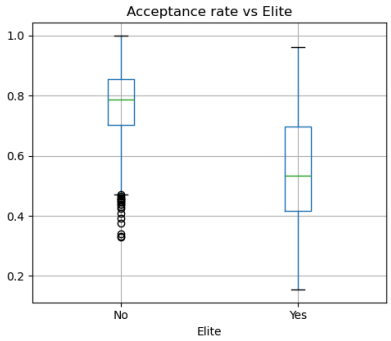
	count	mean	std	min	25%	50%	75%	max
Elite								
No	699.0	0.770	0.122	0.328	0.701	0.788	0.855	1.00
Yes	78.0	0.543	0.193	0.154	0.415	0.533	0.696	0.96

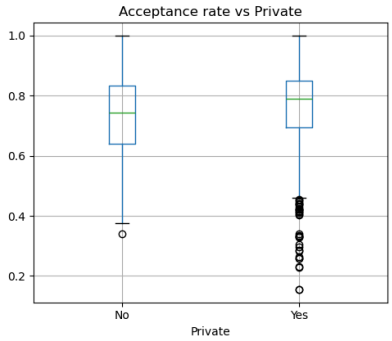
Acceptance rate by Private:

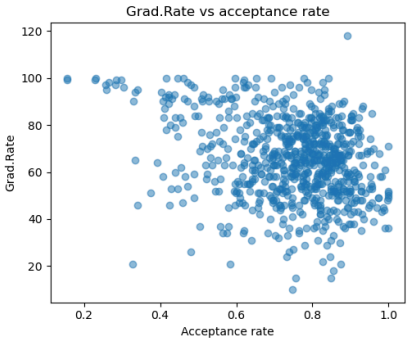
	count	mean	std	min	25%	50%	75%	max
Private								
No	212.0	0.727	0.146	0.340	0.641	0.744	0.834	1.0
Yes	565.0	0.755	0.147	0.154	0.695	0.789	0.851	1.0

Number of colleges by Private x Elite:

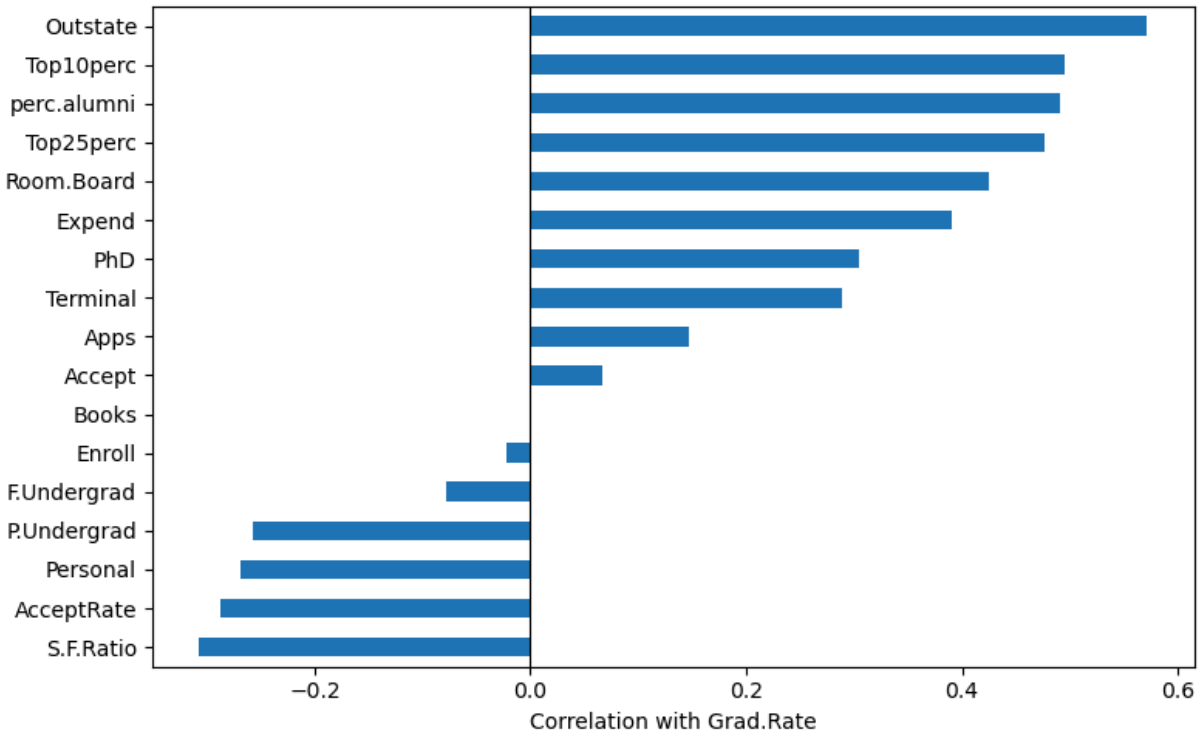
Elite	No	Yes
Private		
No	199	13
Yes	500	65








```
Correlation with Grad.Rate:
S.F.Ratio     -0.307
AcceptRate    -0.287
Personal      -0.269
P.Undergrad   -0.257
F.Undergrad   -0.079
Enroll        -0.022
Books          0.001
Accept         0.067
Apps           0.147
Terminal       0.290
PhD            0.305
Expend         0.390
Room.Board     0.425
Top25perc      0.477
perc.alumni    0.491
Top10perc      0.495
Outstate       0.571
Name: Grad.Rate, dtype: float64
```



Colleges with Grad.Rate > 100:

College	Private	Apps	Grad.Rate
Cazenovia College	Yes	3847	118

In [10]:
```
# Outstate and S.F.Ratio by Private
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
sns.boxplot(x='Private', y='Outstate', data=college, palette='Set2')
plt.title('Out-of-state tuition by type of college')
plt.xlabel('Private')
plt.ylabel('Outstate Tuition ($)')

plt.subplot(1, 2, 2)
sns.boxplot(x='Private', y='S.F.Ratio', data=college, palette='Set2')
plt.title('Student/faculty ratio by type of college')
plt.xlabel('Private')
plt.ylabel('Student-to-Faculty Ratio')

plt.tight_layout()
plt.show()
```

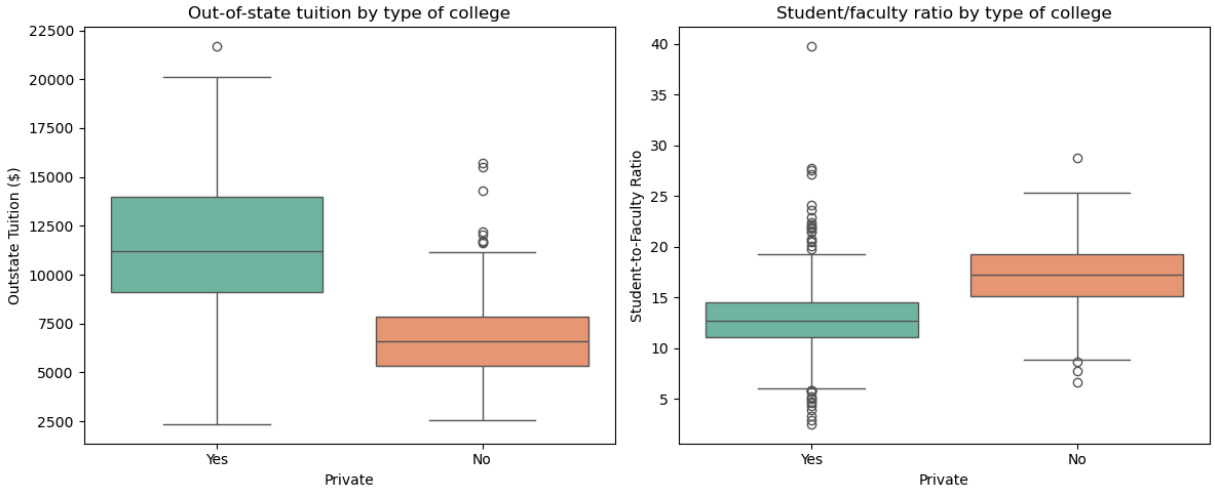
```
/var/folders/ms/wzgt96697xsg5dc8p4zphg4m0000gn/T/ipykernel_37909/841780665.py:5: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

  sns.boxplot(x='Private', y='Outstate', data=college, palette='Set2')
/var/folders/ms/wzgt96697xsg5dc8p4zphg4m0000gn/T/ipykernel_37909/841780665.py:11: FutureWarning:

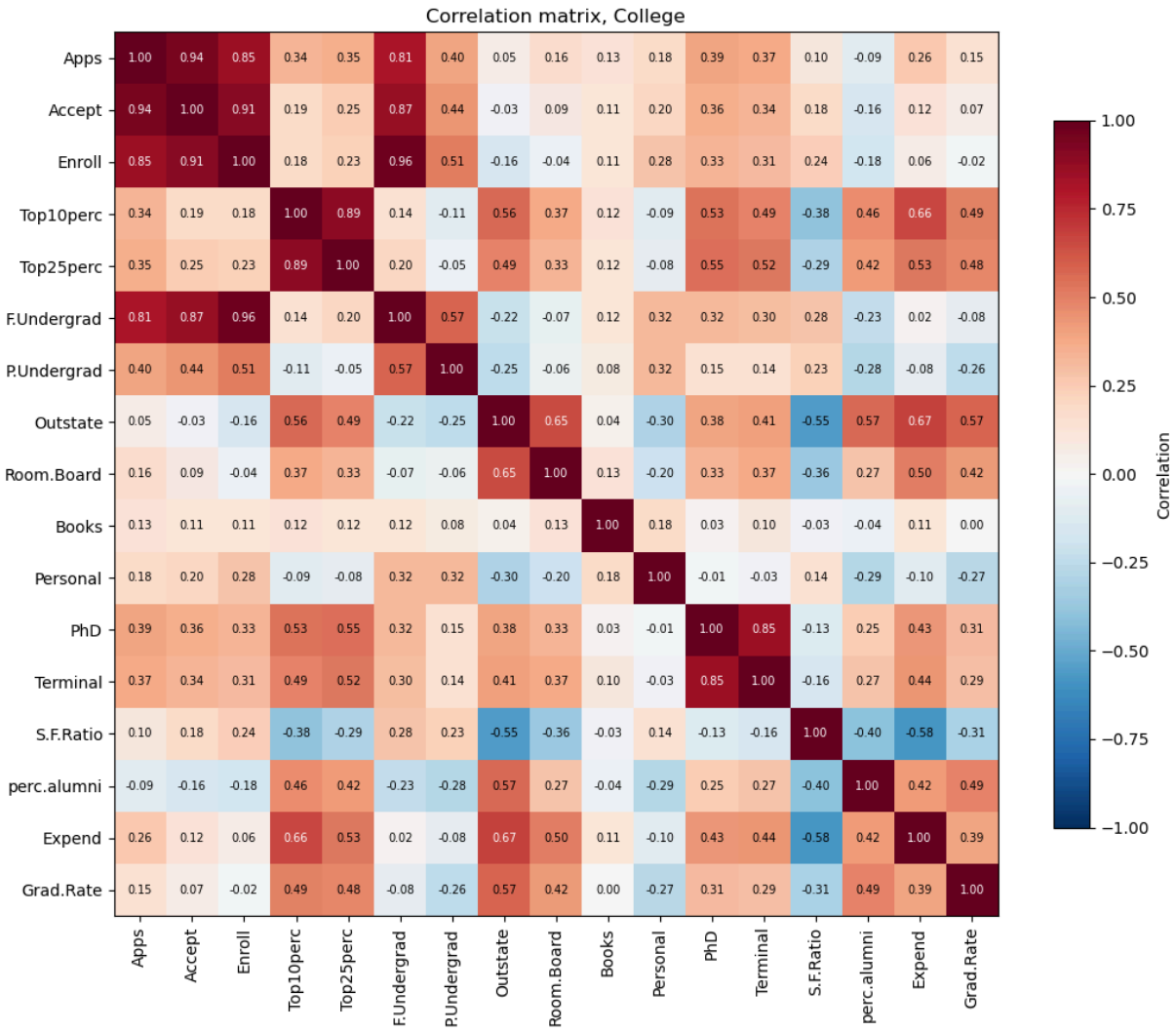
Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

  sns.boxplot(x='Private', y='S.F.Ratio', data=college, palette='Set2')
```



Correlation matrix over all numeric variables.

In [11]:
```
num_cols = ['Apps', 'Accept', 'Enroll', 'Top10perc', 'Top25perc', 'F.Undergr
            'Outstate', 'Room.Board', 'Books', 'Personal', 'PhD', 'Terminal'
            'perc.alumni', 'Expend', 'Grad.Rate']
corr = corr_heatmap(college[num_cols], 'Correlation matrix, College')
strongest_pairs(corr)
```




```
Strongest positive correlations:
Enroll      F.Undergrad    0.965
Apps        Accept         0.943
Accept      Enroll         0.912
Top10perc   Top25perc      0.892
Accept      F.Undergrad    0.874
PhD         Terminal       0.850
Apps        Enroll         0.847
            F.Undergrad    0.814
dtype: float64
Strongest negative correlations:
Room.Board  S.F.Ratio     −0.363
Top10perc   S.F.Ratio     −0.385
S.F.Ratio   perc.alumni   −0.403
Outstate    S.F.Ratio     −0.555
S.F.Ratio   Expend        −0.584
dtype: float64
```

Summary of (h).

- **Elite versus non-elite.** Elite colleges receive more than twice as many applications but accept a much smaller share (acceptance rate 0.54 against 0.77). They charge more (`Outstate` 15 200 against 9 900 dollars), spend about twice as much per

student (`Expend` 18 400 against 8 700), have fewer students per faculty member (10.6 against 14.5), more faculty with a PhD, far fewer part-time students, more alumni who donate (34 % against 21 %) and a much higher graduation rate (83 % against 63 %).

- **Private versus public.** Private colleges are only marginally more selective on average (acceptance rate 0.75 against 0.73) but much more heterogeneous: both the most and the least selective colleges are private. 65 of the 78 elite colleges are private. Private colleges have a lower student/faculty ratio and, as seen in (e), much higher tuition.
- **Graduation rate.** `Grad.Rate` correlates most strongly with `Outstate` (0.57), `Top10perc` (0.50) and `perc.alumni` (0.49), and negatively with `S.F.Ratio` (−0.31), acceptance rate (−0.29), `Personal` (−0.27) and `P.Undergrad` (−0.26). Colleges with well-prepared, full-time students and more resources graduate more of them. These are associations, not causal effects: expensive colleges recruit students who are already likely to graduate.
- **Multicollinearity.** Three groups of variables measure almost the same thing: size (`Apps` , `Accept` , `Enroll` , `F.Undergrad` , correlations 0.81 to 0.97), selectivity (`Top10perc` and `Top25perc` , 0.89) and faculty qualification (`PhD` and `Terminal` , 0.85). In a regression one variable per group would suffice. The strongest negative correlations all involve `S.F.Ratio` , against `Expend` (−0.58) and `Outstate` (−0.56): well-funded colleges have small classes.
- **Data quality.** Cazenovia College has `Grad.Rate` 118 % and Texas A&M at Galveston has `PhD` 103 %; both should be treated as errors.

Exercise 10 – Boston

(a)–(b) Load the data. Each row is a suburb of Boston, each column a variable.

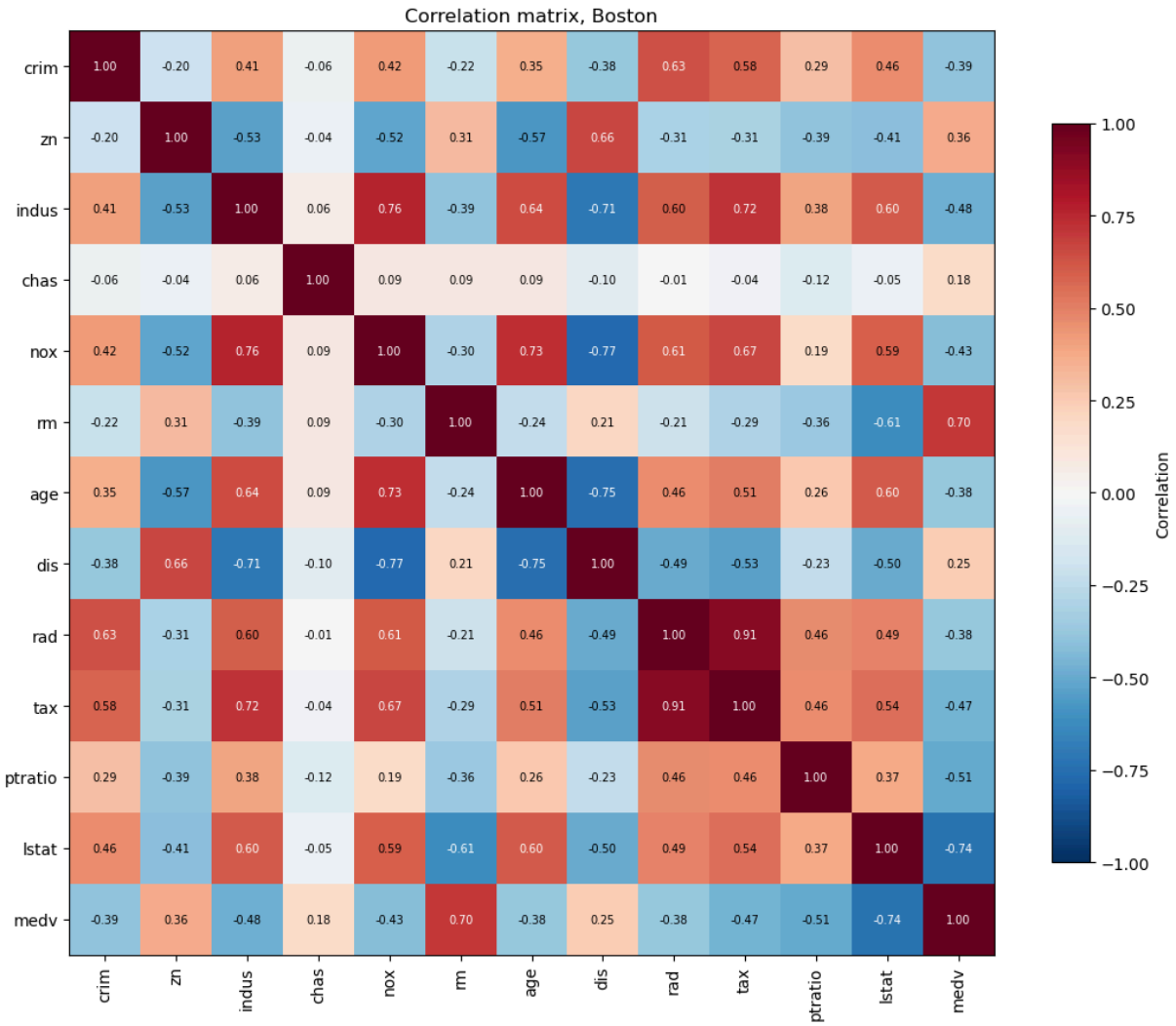
In [12]:
```
Boston = load_data('Boston')
print(f'{Boston.shape[0]} rows (suburbs of Boston) and {Boston.shape[1]} col
Boston.columns
```

```
506 rows (suburbs of Boston) and 13 columns (variables)
```

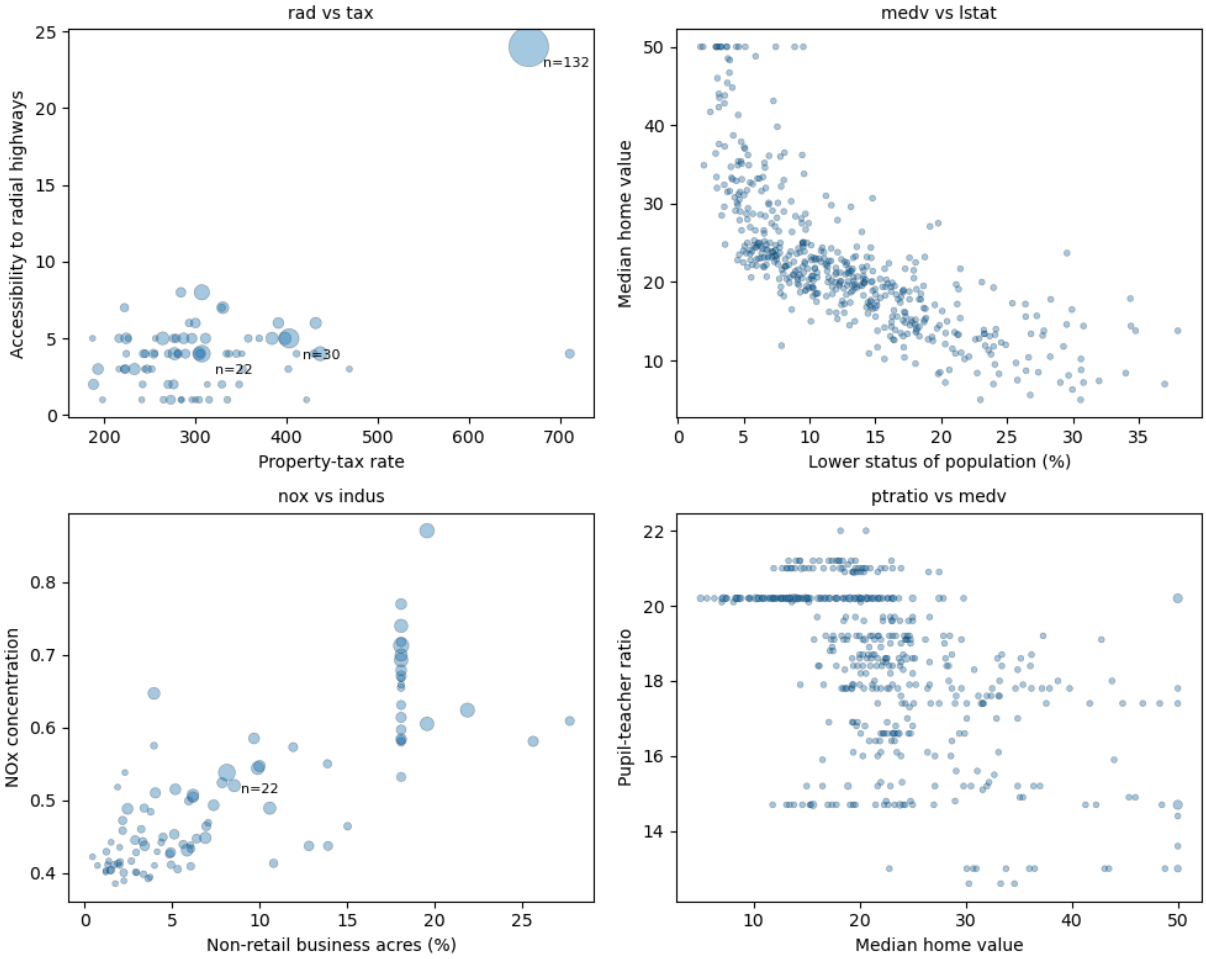
Out[12]:
```
Index(['crim', 'zn', 'indus', 'chas', 'nox', 'rm', 'age', 'dis', 'rad', 'ta
x',
       'ptratio', 'lstat', 'medv'],
      dtype='object')
```

(c) Pairwise relationships between the predictors.

In [13]:
```
corr = corr_heatmap(Boston, 'Correlation matrix, Boston')
```



In [14]:
```
# Point size = number of observations sharing the same (x, y); clusters with
fig, axes = plt.subplots(2, 2, figsize=(10, 8))
pairs = [
    ('tax', 'rad', 'Property-tax rate', 'Accessibility to radial highways'),
    ('lstat', 'medv', 'Lower status of population (%)', 'Median home value'),
    ('indus', 'nox', 'Non-retail business acres (%)', 'NOx concentration'),
    ('medv', 'ptratio', 'Median home value', 'Pupil-teacher ratio'),
]
for ax, (x, y, xlab, ylab) in zip(axes.ravel(), pairs):
    counts = Boston.groupby([x, y]).size().reset_index(name='n')
    ax.scatter(counts[x], counts[y], s=8 + 4*counts['n'], alpha=0.4,
               edgecolor='k', linewidth=0.3)
    for _, r in counts[counts['n'] >= 20].iterrows():
        ax.annotate(f"n={int(r['n'])}", (r[x], r[y]),
                    textcoords='offset points', xytext=(8, -12), fontsize=8)
    ax.set_xlabel(xlab); ax.set_ylabel(ylab)
    ax.set_title(f'{y} vs {x}', fontsize=10)
fig.tight_layout()
plt.show()
```



Comment (c). The correlation matrix and the scatterplots show a few strong structures. `rad` and `tax` are highly correlated (0.91), but the scatterplot reveals why: a single cluster of 132 suburbs shares the values `rad` = 24 and `tax` = 666, so the correlation is driven by one block of suburbs rather than by a smooth relationship. `medv` decreases with `lstat` in a clearly non-linear, convex way, and `medv` is censored at 50, visible as a row of points at the top. `nox` rises with `indus` up to about 18 % non-retail acres, where another large vertical cluster sits (`indus` = 18.1 is the same block of suburbs). `ptratio` shows a weak negative relationship with `medv`, and several horizontal bands appear because the ratio is set per town. Many variables are therefore shared by groups of suburbs, which produces repeated values and clusters rather than continuous clouds.

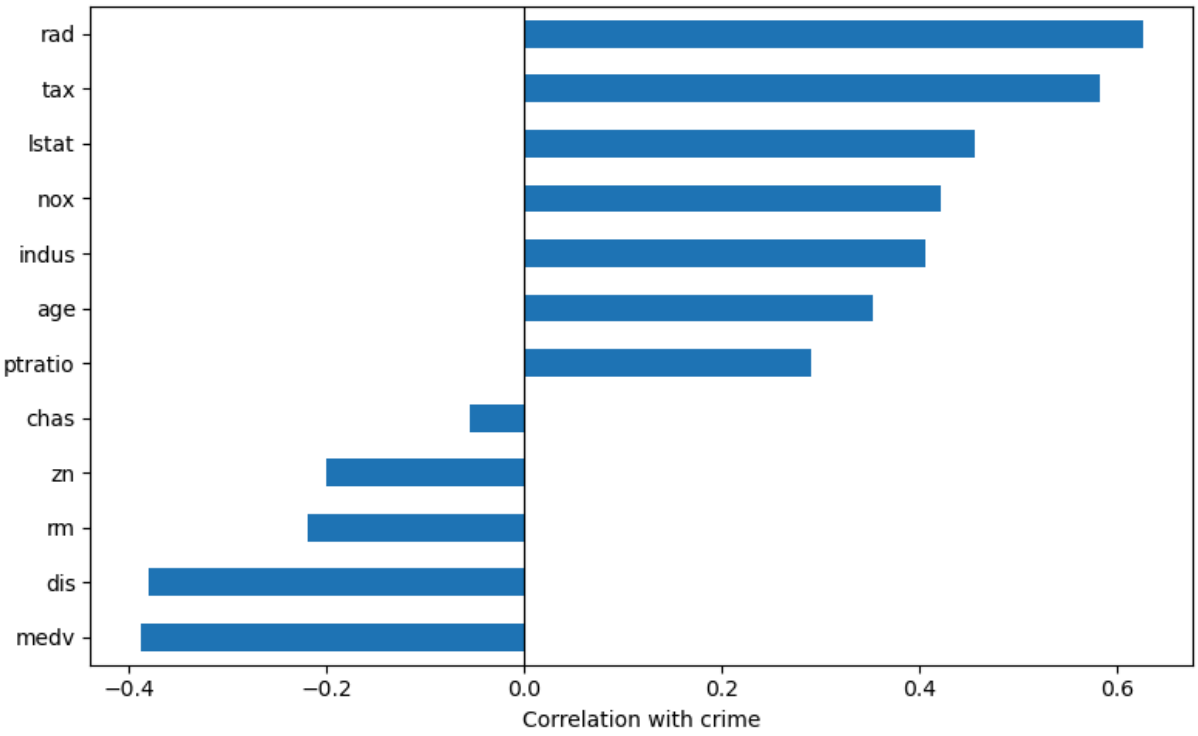
(d) Predictors associated with per capita crime rate.

In [15]:
```
corr_crime = Boston.corr()['crim'].drop('crim').sort_values()
print("Correlation with crime:")
display(corr_crime.round(3))

fig, ax = plt.subplots(figsize=(8, 5))
corr_crime.plot.barh(ax=ax)
ax.set_xlabel('Correlation with crime')
ax.axvline(0, color='black', linewidth=0.8)
```

```
fig.tight_layout()
plt.show()
```

```
Correlation with crime:
medv      -0.388
dis       -0.380
rm        -0.219
zn        -0.200
chas      -0.056
ptratio    0.290
age        0.353
indus      0.407
nox        0.421
lstat      0.456
tax        0.583
rad        0.626
Name: crim, dtype: float64
```



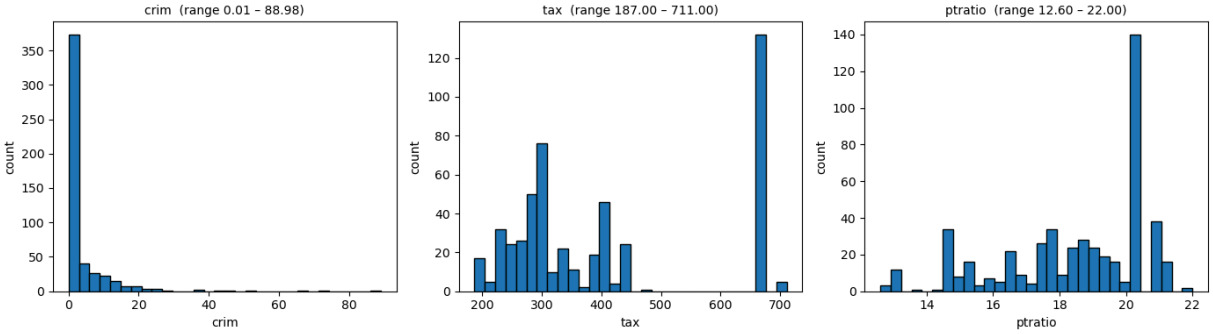
Comment (d). Per capita crime is most strongly associated with `rad` (0.63) and `tax` (0.58): suburbs with high highway accessibility and high property-tax rates have more crime, but this is mostly the block of 132 suburbs with `rad` = 24 and `tax` = 666, which are also the ones with `indus` = 18.1. `lstat` (0.46), `nox` (0.42), `indus` (0.41) and `age` (0.35) are positively associated: older, more industrial, more polluted suburbs with a poorer population have more crime. `medv` (−0.39) and `dis` (−0.38) are negatively associated: crime is lower where homes are more valuable and further from the employment centres. The relationships are far from linear, since `crim` is extremely skewed (see (e)).

(e) Range of `crim`, `tax` and `ptratio`.

In [16]:
```
predictors = ['crim', 'tax', 'ptratio']
fig, axes = plt.subplots(1, 3, figsize=(14, 4))
for ax, col in zip(axes, predictors):
    ax.hist(Boston[col], bins=30, edgecolor='k')
    ax.set_title(f'{col}  (range {Boston[col].min():.2f} – {Boston[col].max(
    ax.set_xlabel(col); ax.set_ylabel('count')
fig.tight_layout()
plt.show()

print(Boston[predictors].describe().round(2))


for col in ['crim', 'tax', 'ptratio']:
    s = Boston[col]
    q1, q3 = s.quantile([0.25, 0.75]); hi = q3 + 1.5*(q3 - q1)
    print(f"{col}: range {s.min():.2f}–{s.max():.2f}, median {s.median():.2f
          f"mean {s.mean():.2f}, IQR outliers {(s > hi).sum()}")
```




```
          crim     tax  ptratio
count   506.00  506.00   506.00
mean      3.61  408.24    18.46
std       8.60  168.54     2.16
min       0.01  187.00    12.60
25%       0.08  279.00    17.40
50%       0.26  330.00    19.05
75%       3.68  666.00    20.20
max      88.98  711.00    22.00
crim: range 0.01–88.98, median 0.26, mean 3.61, IQR outliers 66
tax: range 187.00–711.00, median 330.00, mean 408.24, IQR outliers 0
ptratio: range 12.60–22.00, median 19.05, mean 18.46, IQR outliers 0
```

Comment (e). `crim` ranges from 0.006 to 88.98 with median 0.26 and mean 3.61, so the distribution is extremely right-skewed: half of the suburbs have essentially no crime, but 66 suburbs lie above the IQR outlier limit and a handful have rates above 40. `tax` ranges from 187 to 711 with a clear gap: the bulk lies between 187 and about 470, and then a block of suburbs sits at 666, which is why the histogram is bimodal and no point counts as an outlier. `ptratio` ranges from 12.6 to 22, is left-skewed with a large spike at 20.2, and has no outliers. So yes: some suburbs have particularly high crime rates, a large block has a high tax rate, and pupil-teacher ratios are high but within a narrow range.

(f)–(h) Charles river, median pupil-teacher ratio, suburb with lowest `medv`.

In [17]:
```
# (f)
num_river = int(Boston['chas'].sum())
print(f'{num_river} suburbs bound the Charles River')

# (g)
print(f'Median pupil-teacher ratio: {Boston["ptratio"].median()}')

# (h) Suburb(s) with the lowest median home value, compared with the overall
lowest_medv = Boston[Boston['medv'] == Boston['medv'].min()]
display(lowest_medv)
display(pd.concat([Boston.describe().loc[['min', '25%', '50%', '75%', 'max']
                   lowest_medv.rename(index=lambda i: f'suburb {i}')]).round
```

```
35 suburbs bound the Charles River
Median pupil-teacher ratio: 19.05
```

	crim	zn	indus	chas	nox	rm	age	dis	rad	tax	ptratio	lstat	ı
398	38.3518	0.0	18.1	0	0.693	5.453	100.0	1.4896	24	666	20.2	30.59	
405	67.9208	0.0	18.1	0	0.693	5.683	100.0	1.4254	24	666	20.2	22.98	

	crim	zn	indus	chas	nox	rm	age	dis	rad	tax	ptratio	lstat
min	0.01	0.0	0.46	0.0	0.38	3.56	2.90	1.13	1.0	187.0	12.60	1.73
25%	0.08	0.0	5.19	0.0	0.45	5.89	45.02	2.10	4.0	279.0	17.40	6.95
50%	0.26	0.0	9.69	0.0	0.54	6.21	77.50	3.21	5.0	330.0	19.05	11.36
75%	3.68	12.5	18.10	0.0	0.62	6.62	94.07	5.19	24.0	666.0	20.20	16.96
max	88.98	100.0	27.74	1.0	0.87	8.78	100.00	12.13	24.0	711.0	22.00	37.97
suburb 398	38.35	0.0	18.10	0.0	0.69	5.45	100.00	1.49	24.0	666.0	20.20	30.59
suburb 405	67.92	0.0	18.10	0.0	0.69	5.68	100.00	1.43	24.0	666.0	20.20	22.98

Comment (f)–(h). 35 suburbs bound the Charles river. The median pupil-teacher ratio is 19.05. Two suburbs (index 398 and 405) share the lowest median home value, 5 000 dollars. Both are extreme on almost every predictor: crime rates of 38 and 68 are far above the 75th percentile (3.68), `indus` = 18.1, `nox` = 0.69 and `age` = 100 are all at or above the 75th percentile, `dis` = 1.4 is near the minimum, `rad` = 24 and `tax` = 666 are the maximum, `ptratio` = 20.2 is at the 75th percentile, and `lstat` (31 and 23) is far above the median of 11.4. They are old, dense, polluted, high-crime suburbs close to the employment centres with a poor population, and no residential land zoned for large lots (`zn` = 0).

(i) Suburbs with more than seven / eight rooms per dwelling.

In [18]:
```
# (i) "more than" seven / eight rooms per dwelling
rm_7 = Boston[Boston['rm'] > 7]
```

```
rm_8 = Boston[Boston['rm'] > 8]
print(f'Number of suburbs where the average number of rooms is more than:\n7
display(rm_8)
cmp = pd.DataFrame({
    'all (median)':    Boston.median(),
    'rm > 8 (median)': rm_8.median(),
    'difference':      rm_8.median() - Boston.median(),
}).round(2)
display(cmp)
```

```
Number of suburbs where the average number of rooms is more than:
7: 64
8: 13
```

	crim	zn	indus	chas	nox	rm	age	dis	rad	tax	ptratio	lstat
97	0.12083	0.0	2.89	0	0.4450	8.069	76.0	3.4952	2	276	18.0	4.21
163	1.51902	0.0	19.58	1	0.6050	8.375	93.9	2.1620	5	403	14.7	3.32
204	0.02009	95.0	2.68	0	0.4161	8.034	31.9	5.1180	4	224	14.7	2.88
224	0.31533	0.0	6.20	0	0.5040	8.266	78.3	2.8944	8	307	17.4	4.14
225	0.52693	0.0	6.20	0	0.5040	8.725	83.0	2.8944	8	307	17.4	4.63
226	0.38214	0.0	6.20	0	0.5040	8.040	86.5	3.2157	8	307	17.4	3.13
232	0.57529	0.0	6.20	0	0.5070	8.337	73.3	3.8384	8	307	17.4	2.47
233	0.33147	0.0	6.20	0	0.5070	8.247	70.4	3.6519	8	307	17.4	3.95
253	0.36894	22.0	5.86	0	0.4310	8.259	8.4	8.9067	7	330	19.1	3.54
257	0.61154	20.0	3.97	0	0.6470	8.704	86.9	1.8010	5	264	13.0	5.12
262	0.52014	20.0	3.97	0	0.6470	8.398	91.5	2.2885	5	264	13.0	5.91
267	0.57834	20.0	3.97	0	0.5750	8.297	67.0	2.4216	5	264	13.0	7.44
364	3.47428	0.0	18.10	1	0.7180	8.780	82.9	1.9047	24	666	20.2	5.29

	all (median)	rm > 8 (median)	difference
crim	0.26	0.52	0.26
zn	0.00	0.00	0.00
indus	9.69	6.20	-3.49
chas	0.00	0.00	0.00
nox	0.54	0.51	-0.03
rm	6.21	8.30	2.09
age	77.50	78.30	0.80
dis	3.21	2.89	-0.31
rad	5.00	7.00	2.00
tax	330.00	307.00	-23.00
ptratio	19.05	17.40	-1.65
lstat	11.36	4.14	-7.22
medv	21.20	48.30	27.10

Comment (i). 64 suburbs average more than seven rooms per dwelling and 13 average more than eight. The 13 large-dwelling suburbs are wealthy: the median home value is 48 300 dollars against 21 200 overall, and four of them hit the 50 000 censoring limit. They have much lower `lstat` (4.1 % against 11.4 %), a lower pupil-teacher ratio (17.4 against 19.05), less industry (6.2 against 9.7) and somewhat lower tax. Crime is slightly higher than the overall median (0.52 against 0.26) but still low. One exception is suburb 364, which has `rm` = 8.78 but `medv` only 21.9, with `rad` = 24, `tax` = 666 and `ptratio` = 20.2, so it belongs to the high-tax block and large homes there are not worth as much.

Chapter 3

Exercises: 1, 5, 8, 9

Exercise 8 – Auto, simple linear regression

(a) `mpg` on `horsepower`.

In [19]:
```
auto = load_data('Auto')
X = sm.add_constant(auto['horsepower'])
y = auto['mpg']
lin_reg = sm.OLS(y, X).fit()
print(lin_reg.summary())
```

```python
b0, b1 = lin_reg.params['const'], lin_reg.params['horsepower']
print(f'Predicted mpg at horsepower = 98: {b0 + b1 * 98:.3f}')
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:                      mpg   R-squared:                       0.6
06
Model:                            OLS   Adj. R-squared:                  0.6
05
Method:                 Least Squares   F-statistic:                     59
9.7
Date:                Wed, 23 Sep 2026   Prob (F-statistic):           7.03e-
81
Time:                        13:53:06   Log-Likelihood:                -117
8.7
No. Observations:                 392   AIC:                             236
1.
Df Residuals:                     390   BIC:                             236
9.
Df Model:                           1
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.97
5]
------------------------------------------------------------------------------
const         39.9359      0.717     55.660      0.000      38.525      41.3
47
horsepower    -0.1578      0.006    -24.489      0.000      -0.171      -0.1
45
==============================================================================
Omnibus:                       16.432   Durbin-Watson:                   0.9
20
Prob(Omnibus):                  0.000   Jarque-Bera (JB):               17.3
05
Skew:                           0.492   Prob(JB):                     0.0001
75
Kurtosis:                       3.299   Cond. No.                         32
2.
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is corre
ctly specified.
Predicted mpg at horsepower = 98: 24.467
```

Comment (a).

- *i.* Yes. The coefficient of `horsepower` has t = −24.5 and p-value < 0.0001, so the null hypothesis of no relationship is rejected. The F-statistic of 600 says the same thing.

- *ii.* The relationship is fairly strong: $R^2 = 0.606$, so horsepower alone explains about 61 % of the variance of `mpg`. The residual standard error is 4.9 mpg against a mean `mpg` of 23.4, about 21 %.
- *iii.* Negative. Each additional horsepower is associated with 0.158 fewer miles per gallon on average.
- *iv.* At `horsepower` = 98 the predicted `mpg` is 39.94 − 0.158 × 98 = 24.47. The intervals are computed in the next cell.

Confidence and prediction interval at `horsepower = 98`, computed by hand and checked against statsmodels.

In [20]:
```
def get_residual_se(y_true, y_pred, df):
    # RSS = Sum of squared residual errors
    RSS = ((y_true - y_pred) ** 2).sum()
    s = (RSS / df) ** 0.5
    return s


def get_pred_se(x, x0, CI=True):
    n = len(x)
    x_mean = sum(x) / n
    s_xx = sum((x - x_mean) ** 2)

    if CI:
        inside = (1 / n) + ((x0 - x_mean) ** 2) / s_xx
    else:
        inside = (
            1 + (1 / n) + ((x0 - x_mean) ** 2) / s_xx
        )

    return inside**0.5


def get_CI(x0, ci_level, p, data, x_col, y_col, model):
    n = len(data)
    df = n - p
    alpha = 1 - ci_level

    y_0_hat = model.predict([1.0, x0])[0]
    t_crit = float(t.ppf(1 - alpha / 2, df=df))

    s = get_residual_se(data[y_col], model.fittedvalues, df)
    scale_factor = get_pred_se(data[x_col], x0, CI=True)

    margin = t_crit * s * scale_factor
    return [float(y_0_hat - margin), float(y_0_hat + margin)]


def get_PI(x0, ci_level, p, data, x_col, y_col, model):
    n = len(data)
    df = n - p
    alpha = 1 - ci_level
```

```
    y_0_hat = model.predict([1.0, x0])[0]
    t_crit = float(t.ppf(1 - alpha / 2, df=df))

    s = get_residual_se(data[y_col], model.fittedvalues, df)
    scale_factor = get_pred_se(data[x_col], x0, CI=False)

    margin = t_crit * s * scale_factor
    return [float(y_0_hat - margin), float(y_0_hat + margin)]


# Evaluate at horsepower = 98
x0 = 98
ci = get_CI(
    x0,
    ci_level=0.95,
    p=2,
    data=auto,
    x_col='horsepower',
    y_col='mpg',
    model=lin_reg,
)
pi = get_PI(
    x0,
    ci_level=0.95,
    p=2,
    data=auto,
    x_col='horsepower',
    y_col='mpg',
    model=lin_reg,
)

print(f'95% CI for x0={x0}: {ci}')
print(f'95% PI for x0={x0}: {pi}')

# Check against statsmodels
display(lin_reg.get_prediction(np.array([[1.0, x0]])).summary_frame(alpha=0.
```

```
95% CI for x0=98: [23.97307896070392, 24.961075344320935]
95% PI for x0=98: [14.809396070966717, 34.124758234058135]
```

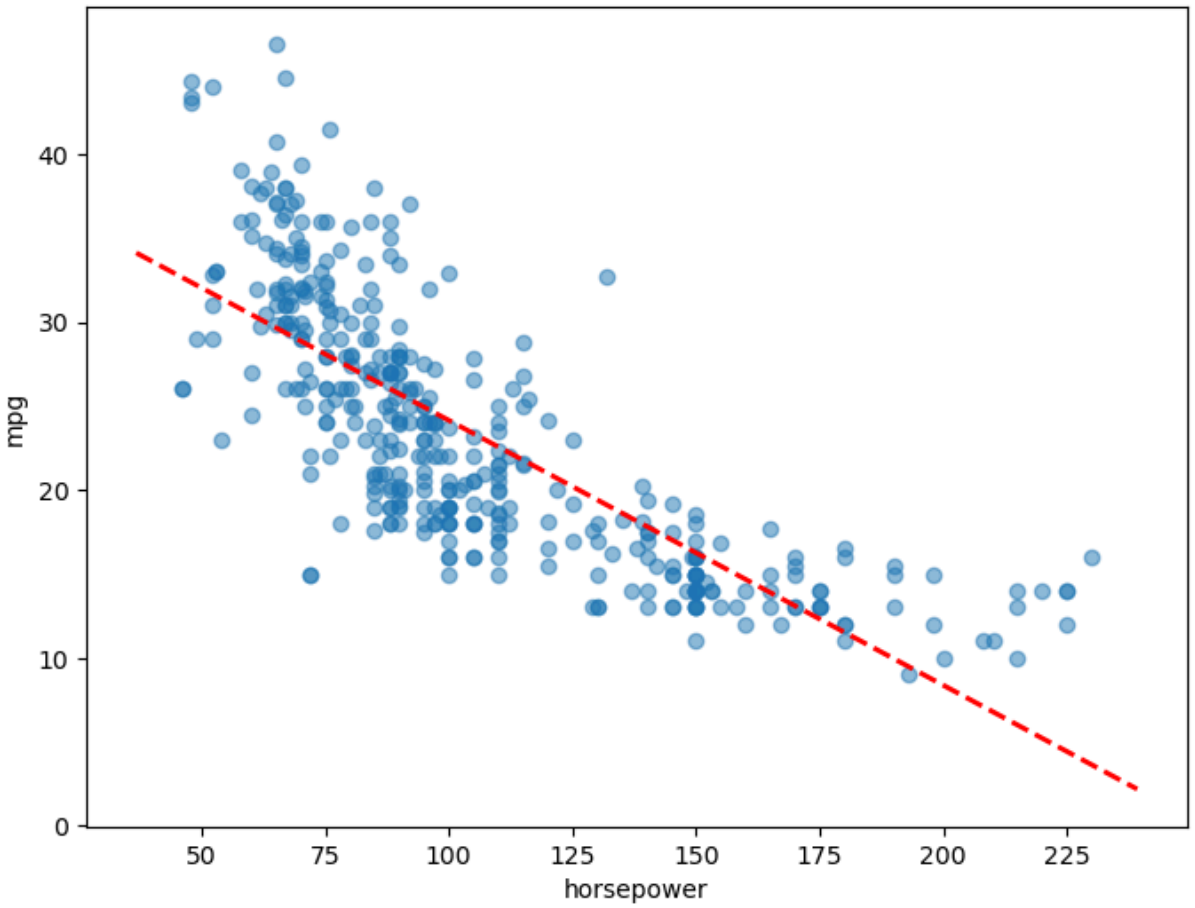
	mean	mean_se	mean_ci_lower	mean_ci_upper	obs_ci_lower	obs_ci_upper
0	24.467	0.251	23.973	24.961	14.809	34.125

Comment (a) iv. The 95 % confidence interval for the mean `mpg` of cars with 98 horsepower is [23.97, 24.96], and the 95 % prediction interval for a single car is [14.81, 34.12]. The hand computation using the formulas $\hat{y}_0 \pm t_{n-2}\, s\sqrt{1/n + (x_0 - \bar{x})^2/S_{xx}}$ and $\hat{y}_0 \pm t_{n-2}\, s\sqrt{1 + 1/n + (x_0 - \bar{x})^2/S_{xx}}$ agrees with statsmodels to all shown decimals. The prediction interval is about ten times wider than the confidence interval because it also includes the irreducible error of an individual car, not only the uncertainty in the fitted line.

(b) Scatterplot with the least squares line.

In [21]:
```
def abline(ax, b, m, *args, **kwargs):
    "Add a line with slope m and intercept b to ax"
    xlim = ax.get_xlim()
    ylim = [m * xlim[0] + b, m * xlim[1] + b]
    ax.plot(xlim, ylim, *args, **kwargs)

fig, ax = plt.subplots(figsize=(8, 6))
ax.scatter(auto['horsepower'], auto['mpg'], alpha=0.5)
abline(ax, lin_reg.params['const'], lin_reg.params['horsepower'], 'r--', lir
ax.set_xlabel('horsepower'); ax.set_ylabel('mpg')
plt.show()
```



Comment (b). The least squares line captures the downward trend, but the points are clearly curved around it: cars with very low and very high horsepower lie mostly above the line, and cars in the middle range (around 100 to 150 hp) lie mostly below it. A straight line is therefore not the right functional form; `mpg` decreases quickly at first and then flattens out.

(c) Diagnostic plots.

In [22]:
```
# Diagnostic plots of the simple regression
resid = lin_reg.resid
fitted = lin_reg.fittedvalues

fig, axes = plt.subplots(2, 2, figsize=(12, 9))
```

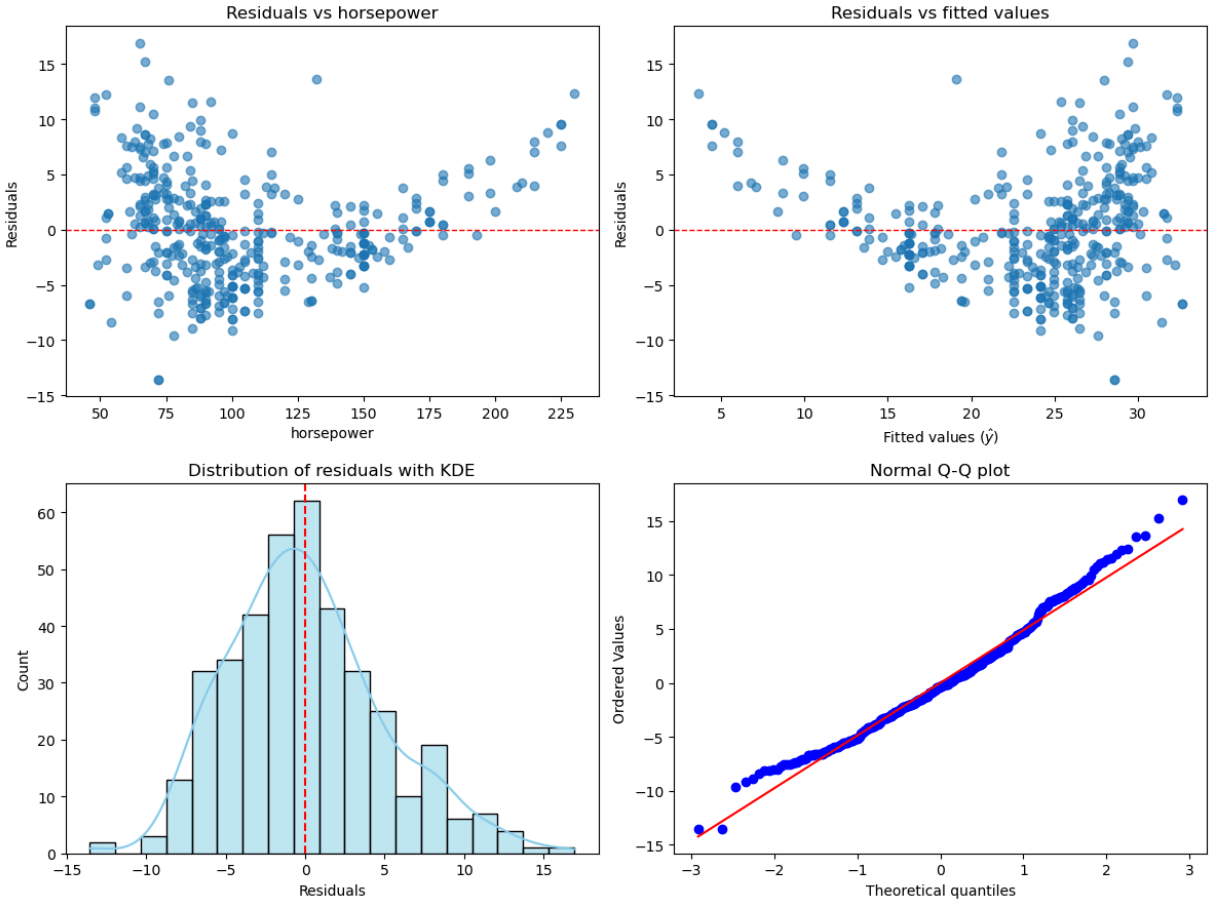
```
ax = axes[0, 0]
ax.scatter(auto['horsepower'], resid, alpha=0.6)
ax.axhline(0, color='red', linestyle='--', linewidth=1)
ax.set(xlabel='horsepower', ylabel='Residuals', title='Residuals vs horsepow

ax = axes[0, 1]
ax.scatter(fitted, resid, alpha=0.6)
ax.axhline(0, color='red', linestyle='--', linewidth=1)
ax.set(xlabel=r'Fitted values ($\hat{y}$)', ylabel='Residuals', title='Resic

ax = axes[1, 0]
sns.histplot(resid, kde=True, ax=ax, color='skyblue', edgecolor='black')
ax.axvline(0, color='red', linestyle='--', linewidth=1.5)
ax.set(xlabel='Residuals', ylabel='Count', title='Distribution of residuals

ax = axes[1, 1]
stats.probplot(resid, dist='norm', plot=ax)
ax.set_title('Normal Q-Q plot')

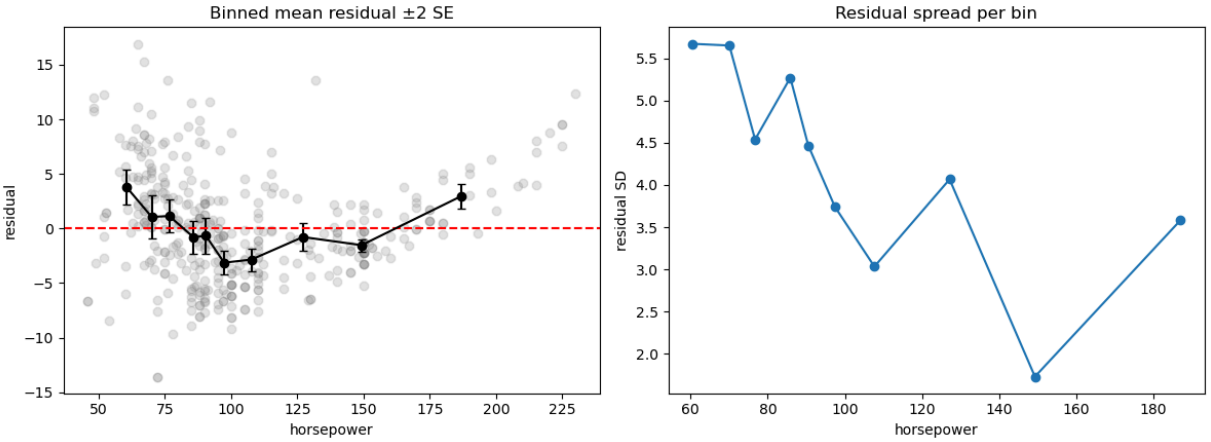
fig.tight_layout()
plt.show()
```



In [23]:
```
# Binned residuals: is the mean residual zero and the spread constant across
x = auto['horsepower']
bins = pd.qcut(x, 10)
g = resid.groupby(bins, observed=True)
summ = pd.DataFrame({'center': x.groupby(bins, observed=True).mean(),
```

```
                        'mean': g.mean(), 'sd': g.std(), 'n': g.size()})
summ['se'] = summ['sd'] / np.sqrt(summ['n'])

fig, axes = plt.subplots(1, 2, figsize=(12, 4.5))
axes[0].scatter(x, resid, alpha=0.2, color='grey')
axes[0].errorbar(summ['center'], summ['mean'], yerr=2*summ['se'], fmt='o-',
axes[0].axhline(0, color='r', ls='--')
axes[0].set(xlabel='horsepower', ylabel='residual', title='Binned mean resic
axes[1].plot(summ['center'], summ['sd'], 'o-')
axes[1].set(xlabel='horsepower', ylabel='residual SD', title='Residual sprea
fig.tight_layout(); plt.show()
```



Comment (c). The residual plots reveal three problems.

- **Non-linearity.** The residuals versus `horsepower` and versus fitted values form a U-shape: positive for low and high horsepower, negative in the middle. The binned means confirm this, being significantly above zero at about 60 hp, significantly below zero between 95 and 150 hp, and above zero again at about 185 hp. A quadratic term or a log transformation of `horsepower` would fix this.
- **Non-constant variance.** The residual spread decreases with horsepower: the standard deviation per bin falls from about 5.6 at 60 hp to about 2 to 3.5 above 130 hp. Since `mpg` is more variable among small cars, a log transformation of `mpg` would stabilise the variance.
- **Non-normality.** The histogram and Q-Q plot show a right-skewed residual distribution with a heavier upper tail (skew 0.49, Jarque-Bera p = 0.0002). This is mild and mostly a consequence of the curvature.

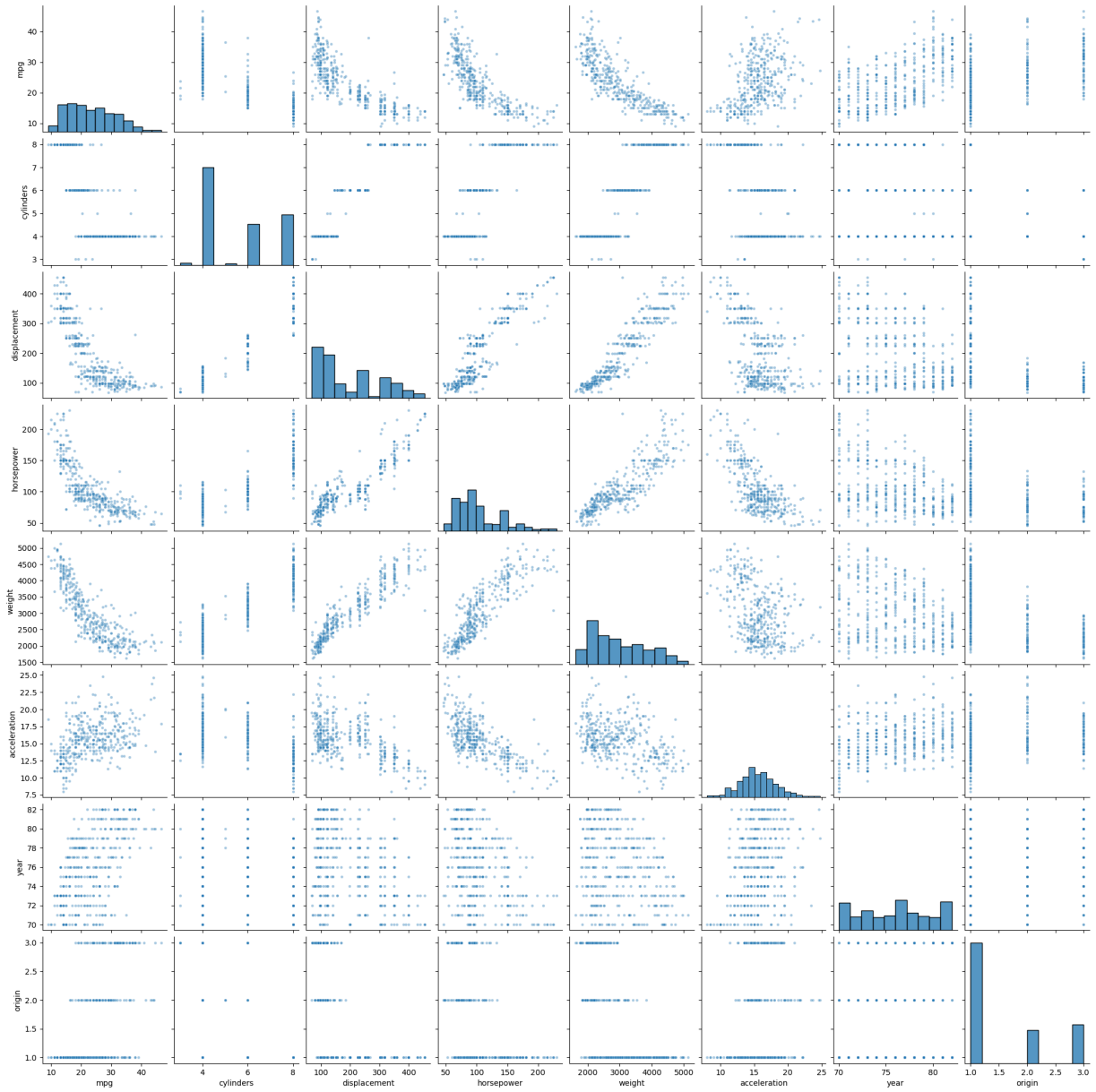
The Durbin-Watson statistic of 0.92 also indicates positive autocorrelation, but this reflects the ordering of the data (cars are roughly sorted by year and make) rather than a time series problem.

Exercise 9 – Auto, multiple linear regression

(a)–(b) Scatterplot matrix and correlation matrix.

In [24]:
```
sns.pairplot(auto.select_dtypes('number'), plot_kws={'alpha': 0.4, 's': 12},
plt.show()

auto.select_dtypes('number').corr().round(2)
```



Out[24]:

	mpg	cylinders	displacement	horsepower	weight	acceleration	year
mpg	1.00	-0.78	-0.81	-0.78	-0.83	0.42	0.58
cylinders	-0.78	1.00	0.95	0.84	0.90	-0.50	-0.35
displacement	-0.81	0.95	1.00	0.90	0.93	-0.54	-0.37
horsepower	-0.78	0.84	0.90	1.00	0.86	-0.69	-0.42
weight	-0.83	0.90	0.93	0.86	1.00	-0.42	-0.31
acceleration	0.42	-0.50	-0.54	-0.69	-0.42	1.00	0.29
year	0.58	-0.35	-0.37	-0.42	-0.31	0.29	1.00
origin	0.57	-0.57	-0.61	-0.46	-0.59	0.21	0.18

Comment (a)–(b). `mpg` is strongly negatively correlated with `weight` (–0.83), `displacement` (–0.81), `cylinders` (–0.78) and `horsepower` (–0.78), and positively with `year` (0.58), `origin` (0.57) and `acceleration` (0.42). The scatterplot matrix shows that all four negative relationships are curved, as in Exercise 8. The four engine-size variables are also very strongly correlated with each other, from 0.84 to 0.95, so they carry almost the same information and multicollinearity is to be expected in the multiple regression. `cylinders`, `year` and `origin` are discrete and appear as vertical bands.

(c) `mpg` on all other variables except `name`.

In [25]:
```
# name is the row index, so dropping mpg leaves all other variables as predi
X = sm.add_constant(auto.drop(columns='mpg'))
y = auto['mpg']
results = sm.OLS(y, X).fit()
print(results.summary())

# F-test for H0: all slopes are zero (relationship between predictors and re
print(f'F = {results.fvalue:.2f}, p = {results.f_pvalue:.2e}')
```

OLS Regression Results

Dep. Variable:	mpg	R-squared:	0.821
Model:	OLS	Adj. R-squared:	0.818
Method:	Least Squares	F-statistic:	252.4
Date:	Wed, 23 Sep 2026	Prob (F-statistic):	2.04e-139
Time:	13:53:08	Log-Likelihood:	-1023.5
No. Observations:	392	AIC:	2063.
Df Residuals:	384	BIC:	2095.
Df Model:	7		
Covariance Type:	nonrobust		

	coef	std err	t	P>|t|	[0.025	0.975]
const	-17.2184	4.644	-3.707	0.000	-26.350	-8.087
cylinders	-0.4934	0.323	-1.526	0.128	-1.129	0.142
displacement	0.0199	0.008	2.647	0.008	0.005	0.035
horsepower	-0.0170	0.014	-1.230	0.220	-0.044	0.010
weight	-0.0065	0.001	-9.929	0.000	-0.008	-0.005
acceleration	0.0806	0.099	0.815	0.415	-0.114	0.275
year	0.7508	0.051	14.729	0.000	0.651	0.851
origin	1.4261	0.278	5.127	0.000	0.879	1.973

Omnibus:	31.906	Durbin-Watson:	1.309
Prob(Omnibus):	0.000	Jarque-Bera (JB):	53.100
Skew:	0.529	Prob(JB):	2.95e-12
Kurtosis:	4.460	Cond. No.	8.59e+04

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is corre

```
ctly specified.
[2] The condition number is large, 8.59e+04. This might indicate that there
are
strong multicollinearity or other numerical problems.
F = 252.43, p = 2.04e-139
```

Comment (c).

- *i.* Yes. The F-statistic is 252 with p-value 2×10^{-139}, so at least one predictor is related to `mpg`. The model explains 82 % of the variance ($R^2 = 0.821$), a large improvement over the 61 % from `horsepower` alone.
- *ii.* `weight` ($p < 0.001$), `year` ($p < 0.001$), `origin` ($p < 0.001$) and `displacement` ($p = 0.008$) are statistically significant. `cylinders`, `horsepower` and `acceleration` are not, although `horsepower` was highly significant on its own. This is the multicollinearity seen in (b): once `weight` and `displacement` are in the model, `horsepower` and `cylinders` add little independent information, which is also flagged by the large condition number. The sign of `displacement` is even positive, which should not be interpreted causally.
- *iii.* The coefficient of `year` is 0.75: holding the other variables fixed, each model year is associated with 0.75 more miles per gallon, so fuel efficiency improved by about 7.5 mpg over the decade 1970 to 1982 for cars of the same size and power. `origin` is coded 1 = American, 2 = European, 3 = Japanese and is treated as numeric here; its coefficient of 1.43 says that European and Japanese cars get more mpg than American cars of the same specification.

(d) Diagnostic plots (the four plots of R's `plot(lm)`).

In [26]:
```python
def diagnostic_plots(results, n_label=3):
    """The four plots of R's plot(lm) for a fitted statsmodels OLS result."""
    infl = results.get_influence()
    fitted, resid = results.fittedvalues, results.resid
    std_resid = infl.resid_studentized_internal
    leverage, cooks = infl.hat_matrix_diag, infl.cooks_distance[0]
    p, n = results.df_model + 1, len(resid)
    idx = results.model.data.row_labels
    fig, axes = plt.subplots(2, 2, figsize=(12, 9))

    # 1. Residuals vs fitted: curvature = wrong functional form
    ax = axes[0, 0]
    ax.scatter(fitted, resid, alpha=0.5)
    s = lowess(resid, fitted, frac=0.6); ax.plot(s[:, 0], s[:, 1], 'r-')
    ax.axhline(0, color='grey', ls='--')
    ax.set(xlabel='Fitted values', ylabel='Residuals Fitted')
    for i in np.argsort(np.abs(resid.values))[-n_label:]:
        ax.annotate(idx[i], (fitted.iloc[i], resid.iloc[i]), fontsize=8)

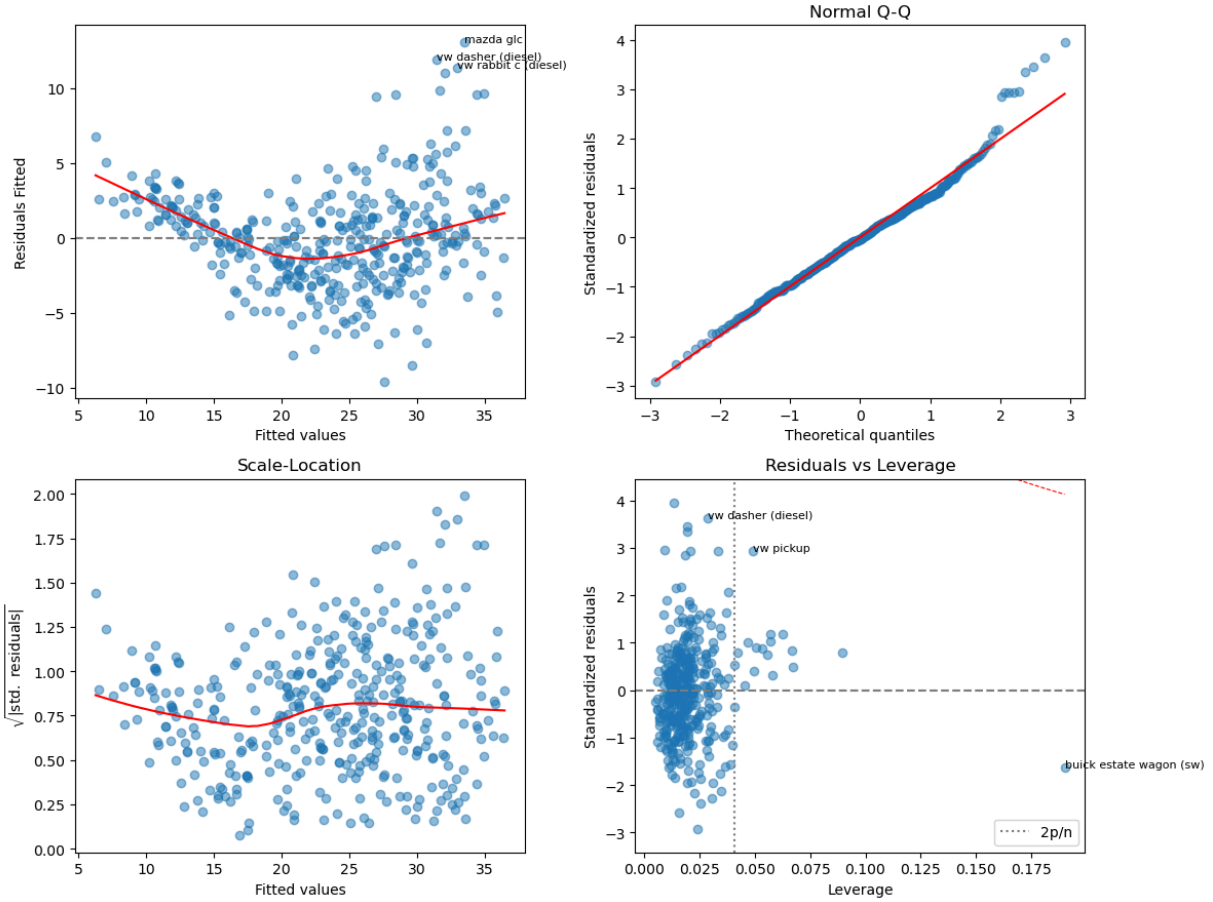
    # 2. Normal Q-Q of standardized residuals
    ax = axes[0, 1]
    (osm, osr), (slope, intercept, _) = stats.probplot(std_resid, dist='norm
    ax.scatter(osm, osr, alpha=0.5); ax.plot(osm, slope*osm + intercept, 'r-
```

```
    ax.set(xlabel='Theoretical quantiles', ylabel='Standardized residuals',

    # 3. Scale-location: trend = non-constant v
    ax = axes[1, 0]
    sqrt_abs = np.sqrt(np.abs(std_resid))
    ax.scatter(fitted, sqrt_abs, alpha=0.5)
    s = lowess(sqrt_abs, fitted, frac=0.6); ax.plot(s[:, 0], s[:, 1], 'r-')
    ax.set(xlabel='Fitted values', ylabel=r'$\sqrt{|\mathrm{std.\ residuals}}

    # 4. Residuals vs leverage with Cook's distance contours
    ax = axes[1, 1]
    ax.scatter(leverage, std_resid, alpha=0.5)
    ax.axhline(0, color='grey', ls='--')
    ax.axvline(2*p/n, color='grey', ls=':', label='2p/n')
    lev = np.linspace(leverage.min(), leverage.max(), 100)
    for d in (0.5, 1.0):
        for sign in (1, -1):
            ax.plot(lev, sign*np.sqrt(d*p*(1 - lev)/lev), 'r--', lw=0.8)
    ax.set_ylim(std_resid.min() - 0.5, std_resid.max() + 0.5)
    ax.set(xlabel='Leverage', ylabel='Standardized residuals', title="Residu
    for i in np.argsort(cooks)[-n_label:]:
        ax.annotate(idx[i], (leverage[i], std_resid[i]), fontsize=8)
    ax.legend()
    fig.tight_layout()
    return fig

diagnostic_plots(results)
plt.show()
```



Comment (d). The four diagnostic plots show the same non-linearity as in the simple regression: the residuals versus fitted values are U-shaped, with large positive residuals for the most efficient cars (fitted values above 30). The scale-location plot shows a mildly increasing spread with the fitted value. The Q-Q plot is close to the line except in the upper tail, where about ten standardized residuals lie above 2.9; the largest are `mazda glc` (3.95), `vw dasher (diesel)` (3.63) and `vw rabbit c (diesel)` (3.45), all small, mostly diesel cars whose fuel efficiency the linear model underestimates. These are unusually large residuals but none exceeds 4, so they are moderate outliers. In the leverage plot, `buick estate wagon (sw)` has by far the highest leverage, 0.19 against the threshold 2p/n = 0.04, because it combines the heaviest weight with a large engine; its standardized residual is only −1.6 and its Cook's distance 0.08, so it is a high-leverage point but not an influential one. `vw pickup` and `vw dasher (diesel)` combine leverage slightly above the threshold with large residuals, but their Cook's distances are below 0.06, so no single observation drives the fit.

(e) Interaction effects.

In [27]:
```
# Interaction terms added one at a time to the full model
base = 'mpg ~ cylinders + displacement + horsepower + weight + acceleration
for inter in ['horsepower:weight', 'displacement:horsepower', 'horsepower:ye
              'weight:year', 'cylinders:displacement', 'year:origin']:
    m = ols(base + ' + ' + inter, auto).fit()
    print(f'{inter:24s} coef={m.params[inter]: .3e}  p={m.pvalues[inter]:.1e

m_hw = ols('mpg ~ horsepower * weight + year + origin', auto).fit()
print(m_hw.summary())
```

```
horsepower:weight        coef= 5.529e-05  p=4.1e-23  adjR2=0.8590
displacement:horsepower  coef= 5.236e-04  p=3.4e-24  adjR2=0.8608
horsepower:year          coef=-1.139e-02  p=6.7e-17  adjR2=0.8481
weight:year              coef=-4.879e-04  p=1.5e-14  adjR2=0.8439
cylinders:displacement   coef= 1.361e-02  p=2.8e-14  adjR2=0.8433
year:origin              coef= 1.989e-01  p=1.1e-03  adjR2=0.8228
                            OLS Regression Results
==============================================================================
==
Dep. Variable:                     mpg   R-squared:                       0.8
61
Model:                             OLS   Adj. R-squared:                  0.8
59
Method:                  Least Squares   F-statistic:                     47
7.5
Date:                 Wed, 23 Sep 2026   Prob (F-statistic):          8.32e-1
63
Time:                         13:53:08   Log-Likelihood:                -974.
68
No. Observations:                 392   AIC:                             196
1.
Df Residuals:                     386   BIC:                             198
5.
Df Model:                           5
Covariance Type:            nonrobust
==============================================================================
==========
                           coef    std err          t      P>|t|      [0.025
0.975]
------------------------------------------------------------------------------
---------
Intercept              0.8145      3.969      0.205      0.838      -6.990
8.619
horsepower            -0.2160      0.021    -10.514      0.000      -0.256
-0.176
weight                -0.0111      0.001    -17.435      0.000      -0.012
-0.010
horsepower:weight  5.501e-05   5.05e-06     10.891      0.000    4.51e-05
6.49e-05
year                   0.7677      0.045     17.195      0.000       0.680
0.855
origin                 0.7224      0.233      3.103      0.002       0.265
1.180
==============================================================================
==
Omnibus:                       40.135   Durbin-Watson:                   1.4
51
Prob(Omnibus):                  0.000   Jarque-Bera (JB):                69.9
58
Skew:                           0.627   Prob(JB):                     6.44e-
16
Kurtosis:                       4.646   Cond. No.                     1.08e+
07
==============================================================================
==
```

```
Notes:
[1] Standard Errors assume that the covariance matrix of the errors is corre
ctly specified.
[2] The condition number is large, 1.08e+07. This might indicate that there
are
strong multicollinearity or other numerical problems.
```

Comment (e). Every interaction tried is statistically significant, which is not surprising with 392 observations. The two most useful are `horsepower:weight` and `displacement:horsepower`, which each raise the adjusted R^2 from 0.818 to about 0.86. In the model `mpg ~ horsepower * weight + year + origin` all terms are significant, and the interaction coefficient is positive (5.5×10^{-5}, $p < 0.001$): the penalty of extra horsepower on `mpg` is smaller for heavy cars than for light cars, or equivalently the effect of weight is weaker for powerful cars. This model with five terms has a higher adjusted R^2 (0.859) than the full additive model with seven predictors, and it drops the collinear `cylinders`, `displacement` and `acceleration`. The residuals remain right-skewed, so the interaction does not remove the non-linearity completely.

(f) Transformations of the variables.

In [28]:
```
# Transformations of the variables: adjusted R^2 and Breusch-Pagan test for
models = {
  'base':        base,
  'hp^2':        base.replace('horsepower', 'horsepower + I(horsepower**2)')
  'log(hp)':     base.replace('horsepower', 'np.log(horsepower)'),
  'sqrt(hp)':    base.replace('horsepower', 'np.sqrt(horsepower)'),
  'log(mpg)':    'np.log(mpg) ~ cylinders + displacement + horsepower + weig
  'log-log':     'np.log(mpg) ~ cylinders + displacement + np.log(horsepower
}
for name, f in models.items():
    m = ols(f, auto).fit()
    print(f'{name:10s} adjR2={m.rsquared_adj:.4f}  BP p={het_breuschpagan(m.
```

```
base       adjR2=0.8182  BP p=1.2e-04
hp^2       adjR2=0.8522  BP p=4.5e-06
log(hp)    adjR2=0.8340  BP p=2.4e-05
sqrt(hp)   adjR2=0.8237  BP p=1.1e-04
log(mpg)   adjR2=0.8773  BP p=9.2e-02
log-log    adjR2=0.8885  BP p=1.3e-01
```

Comment (f). Transforming `horsepower` improves the fit: a quadratic term raises the adjusted R^2 from 0.818 to 0.852, log(`horsepower`) to 0.834 and the square root to 0.824. All of these still fail the Breusch-Pagan test ($p < 0.001$), so the variance of the residuals remains non-constant. Transforming the response works better: log(`mpg`) gives adjusted R^2 0.877 and a Breusch-Pagan p-value of 0.09, and the log-log model with log(`horsepower`) and log(`weight`) gives the best fit, adjusted R^2 0.889, with $p = 0.13$ for the Breusch-Pagan test, so heteroscedasticity is no longer detected. The log transformation of `mpg` makes sense physically, since fuel consumption per mile (1/mpg) is closer to proportional to weight and power, and it turns the multiplicative relationship

into an additive one. The recommended model is therefore log(`mpg`) on log(`horsepower`), log(`weight`), `year` and `origin` .

Additional coding task: bias-variance trade-off

Setup

The ground truth is

$$f(x) = \sin(2x_1), \qquad y = f(x) + \varepsilon, \qquad \varepsilon \sim \mathcal{N}(0, \sigma^2),\ \sigma = 0.5.$$

There are p predictors $x = (x_1, \ldots, x_p)$, each drawn independently from $\mathcal{U}(-1, 1)$, but only x_1 carries information about y; the remaining $p - 1$ predictors are pure noise. We use a fixed design with $n_{\text{train}} = 50$ training points and $n_{\text{test}} = 500$ noise-free test points $\{(x_0, f(x_0))\}$, and vary $p \in \{1, 2, 3, 4, 10, 20\}$.

Two estimators $\hat{f}$ are compared:

- **Linear regression** $\hat{f}\,(x) = \hat{\beta}_0 + \sum_{j=1}^{p} \hat{\beta}_j x_j$, fitted by least squares.
- **KNN regression** $\hat{f}\,(x_0) = \frac{1}{K} \sum_{i \in N_K(x_0)} y_i$, where $N_K(x_0)$ are the K training points closest to x_0 in Euclidean distance over all p predictors, for $K = 1, \ldots, 9$.

Bias-variance decomposition

For a fixed test point x_0 and a training set $\mathcal{D}$ (here: fixed X, random noise ε), the expected squared error of $\hat{f}$ against the *true* function decomposes as

$$\mathbb{E}_{\mathcal{D}}\Big[\big(f(x_0) - \hat{f}\,(x_0; \mathcal{D})\big)^2\Big] = \underbrace{\big(f(x_0) - \mathbb{E}_{\mathcal{D}}[\hat{f}\,(x_0)]\big)^2}_{\text{Bias}^2(x_0)} + \underbrace{\mathbb{E}_{\mathcal{D}}\Big[\big(\hat{f}\,(x_0) - \mathbb{E}_{\mathcal{D}}[\hat{f}\,(x_0)]\big)^2\Big]}_{\text{Var}(x_0)}.$$

The cross term vanishes because $\mathbb{E}_{\mathcal{D}}\big[\hat{f}\,(x_0) - \mathbb{E}_{\mathcal{D}}[\hat{f}\,(x_0)]\big] = 0$. If the test response were observed with noise, the irreducible error $\sigma^2 = 0.25$ would be added; since we compare with $f(x_0)$ directly it is left out, so $\text{MSE} = \text{Bias}^2 + \text{Var}$ holds exactly.

Monte Carlo estimation

The expectation over $\mathcal{D}$ is approximated by drawing $L = 100$ independent noise vectors $\varepsilon^{(l)}$, refitting the model on $y^{(l)} = f(X) + \varepsilon^{(l)}$ each time, and predicting at every test point. Writing $\hat{f}^{(l)}(x_0)$ for the prediction from replication l,

$$\bar{f}\,(x_0) = \frac{1}{L} \sum_{l=1}^{L} \hat{f}^{(l)}(x_0)$$

estimates $\mathbb{E}_{\mathcal{D}}[\hat{f}(x_0)]$. Averaging over the n_{test} test points gives the three reported quantities:

$$\widehat{\text{Bias}^2} = \frac{1}{n_{\text{test}}} \sum_{x_0} \left(f(x_0) - \bar{f}(x_0)\right)^2, \qquad \widehat{\text{Var}} = \frac{1}{n_{\text{test}}} \sum_{x_0} \frac{1}{L} \sum_{l=1}^{L} \left(\hat{f}^{(l)}(x_0) - \bar{f}(x_0)\right)^2$$

In the code, `yhat_lm` and `yhat_knn[:, :, j]` are the $n_{\text{test}} \times L$ matrices of $\hat{f}^{(l)}(x_0)$, `ybar_*` is $\bar{f}(x_0)$, `biasSQ_*`, `variance_*` and `err_*` are the three estimates above, and the results are collected in `out_lm` (one row per p) and `out_knn` (one row per (p, K)).

What to expect

- **KNN, effect of K.** Small K averages few noisy responses, so $\text{Var} \approx \sigma^2/K$ is large and the bias is small: a flexible fit. Large K averages over a wide neighbourhood in which f is not constant, so the bias grows while the variance shrinks. The MSE is U-shaped in $1/K$.
- **KNN, effect of p.** With more noise predictors the Euclidean distance is dominated by irrelevant coordinates, so the K nearest neighbours are no longer close in x_1 and the local average is taken over points with very different f values. This inflates the bias for every K: the curse of dimensionality.
- **Linear regression.** $\hat{f}$ is linear but f is a sine, so there is a model bias that does not depend on p. Each noise predictor adds one coefficient estimated from 50 points, so the variance grows roughly linearly in p, approximately $\sigma^2(p+1)/n_{\text{train}}$.
- The MSE pattern should resemble FIGURE 3.20 in the book: KNN is competitive for $p \leq 2$ and linear regression wins for larger p.

In [29]:
```
import numpy as np
import pandas as pd
from sklearn.neighbors import KNeighborsRegressor
from sklearn.linear_model import LinearRegression

ntrain = 50  # Training
ntest = 500  # Test
nrep = 100  # Repeat 100 times
p = 20
puse = np.array([1, 2, 3, 4, 10, 20])  # Number of predictors
k = np.arange(1, 10)

sigma = 0.5
np.random.seed(0)
Xtrain = np.random.uniform(-1, 1, [ntrain, p])
Xtest = np.random.uniform(-1, 1, [ntest, p])
y0 = np.sin(2 * Xtrain[:, 0])  # Only the first predictor is related to y
ytest = np.sin(2 * Xtest[:, 0])

out_knn = pd.DataFrame()  # Output results
out_lm = pd.DataFrame()
```

```
for i in range(len(puse)):
    yhat_lm = np.zeros((ntest, nrep))
    yhat_knn = np.zeros((ntest, nrep, len(k)))

    for l in range(nrep):
        y = y0 + np.random.normal(0, sigma, ntrain)

        ######### DO: fit linear regression using LinearRegression funciton,
        fit_lm = LinearRegression().fit(Xtrain[:, :puse[i]], y)
        yhat_lm[:, l] =  fit_lm.predict(Xtest[:, :puse[i]])

        for j in range(len(k)):

            ######### DO: fit knn using KNeighborsRegressor funciton, assign
            fit_knn = KNeighborsRegressor(k[j]).fit(Xtrain[:, :puse[i]], y)
            yhat_knn[:, l, j] = fit_knn.predict(Xtest[:, :puse[i]])


        #print(i + 1, "-th p, ", l + 1, "-th repetition finished. \n")


    ######### DO: compute bias and variance of linear regression #########

    # Compute mean of predicted values
    ybar_lm = yhat_lm.mean(axis=1)

    # Compute bias^2
    biasSQ_lm = np.mean((ytest - ybar_lm)**2)

    # Compute variance
    variance_lm = np.mean((yhat_lm - ybar_lm[:, None])**2)

    # Compute total MSE
    err_lm = np.mean((yhat_lm - ytest[:, None])**2)

    out_lm = pd.concat([out_lm, pd.DataFrame({'biasSQ_lm': [biasSQ_lm], 'var
    out_lm = out_lm.reset_index(drop=True)

    ######### DO: compute bias and variance of knn regression #########
    for j in range(len(k)):
        yh = yhat_knn[:, :, j]
        # Compute mean of predicted values
        ybar_knn = yh.mean(axis=1)

        # Compute bias^2
        biasSQ_knn = np.mean((ytest - ybar_knn)**2)


        # Compute variance
        variance_knn = np.mean((yh - ybar_knn[:, None])**2)

        # Compute total MSE
        err_knn = np.mean((yh - ytest[:, None])**2)
```

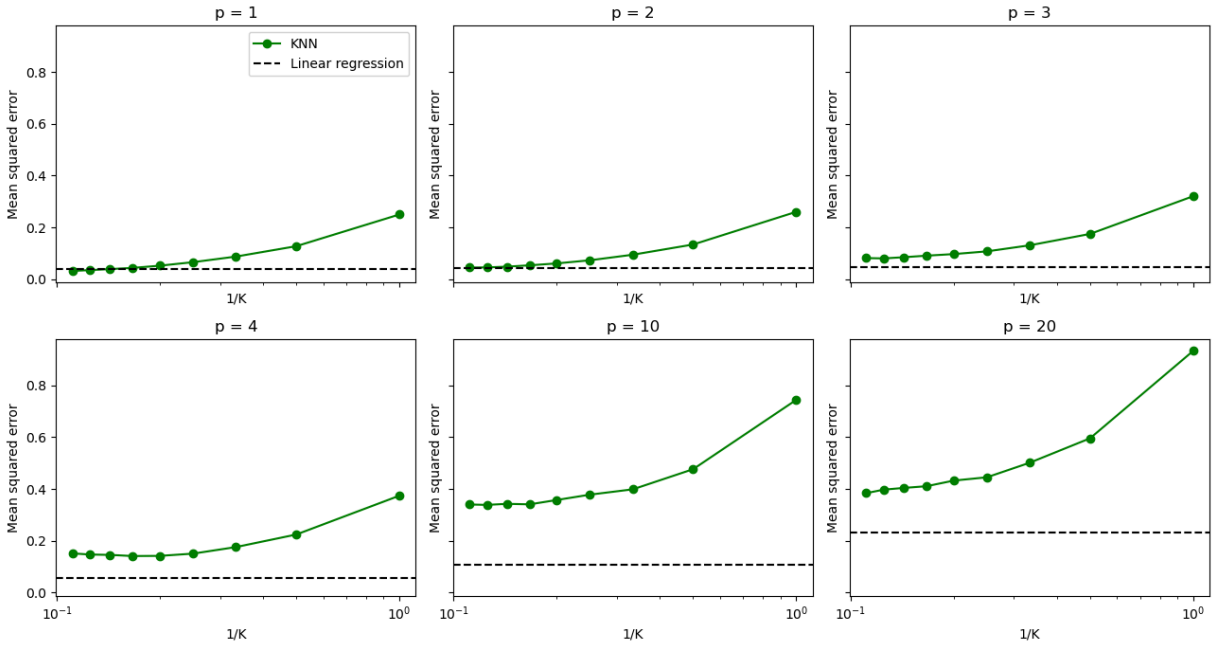
```
        out_knn = pd.concat([out_knn, pd.DataFrame({'biasSQ_knn': [biasSQ_kr
        out_knn = out_knn.reset_index(drop=True)
```

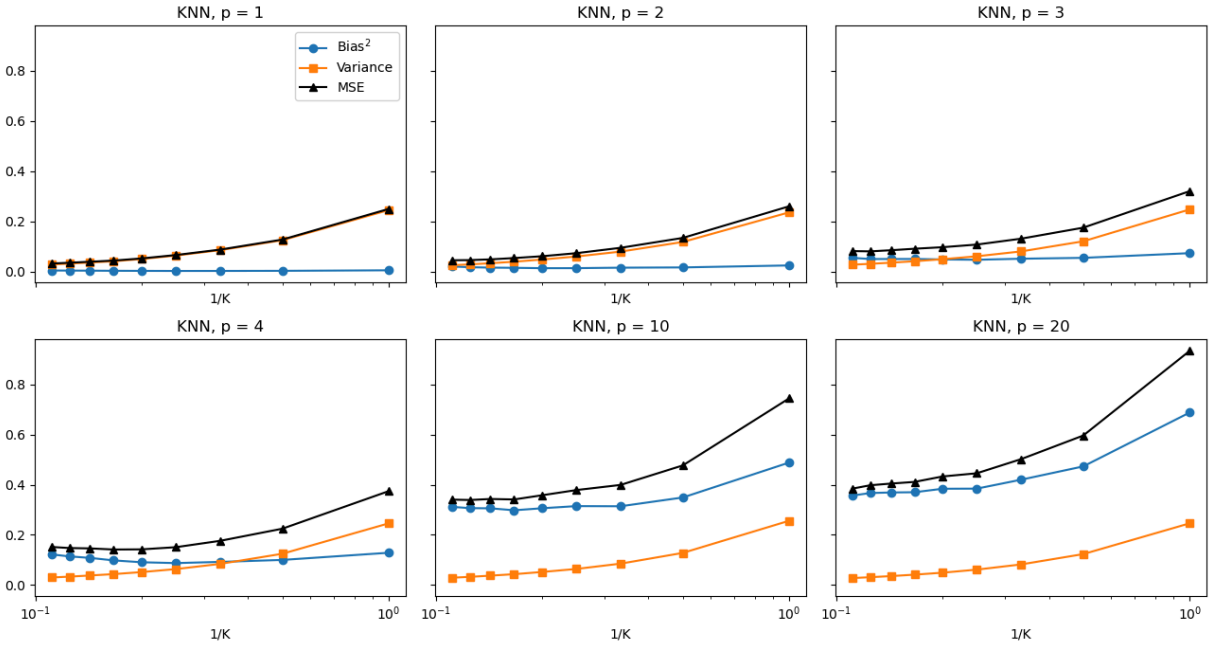
In [30]:

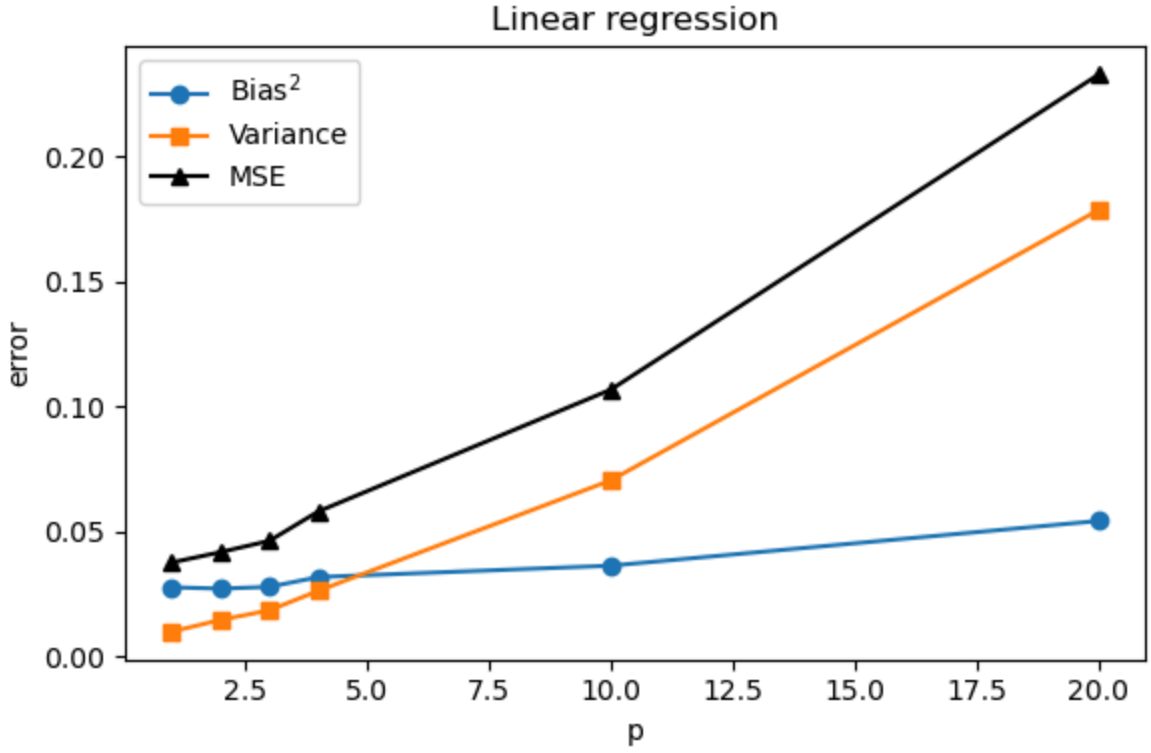
```
fig, axes = plt.subplots(2, 3, figsize=(13, 7), sharex=True, sharey=True)
for ax, pv in zip(axes.ravel(), puse):
    d = out_knn[out_knn['p'] == pv].sort_values('1/K')
    lm = out_lm.loc[out_lm['p'] == pv, 'MSE'].iloc[0]
    ax.plot(d['1/K'], d['MSE'], 'o-', color='green', label='KNN')
    ax.axhline(lm, color='k', ls='--', label='Linear regression')
    ax.set_xscale('log'); ax.set_title(f'p = {pv}')
    ax.set_xlabel('1/K'); ax.set_ylabel('Mean squared error')
axes[0, 0].legend(); fig.tight_layout(); plt.show()


fig, axes = plt.subplots(2, 3, figsize=(13, 7), sharex=True, sharey=True)
for ax, pv in zip(axes.ravel(), puse):
    d = out_knn[out_knn['p'] == pv].sort_values('1/K')
    ax.plot(d['1/K'], d['biasSQ_knn'], 'o-', label='Bias$^2$')
    ax.plot(d['1/K'], d['variance_knn'], 's-', label='Variance')
    ax.plot(d['1/K'], d['MSE'], 'k^-', label='MSE')
    ax.set_xscale('log'); ax.set_title(f'KNN, p = {pv}'); ax.set_xlabel('1/K
axes[0, 0].legend(); fig.tight_layout(); plt.show()


fig, ax = plt.subplots(figsize=(6, 4))
ax.plot(out_lm['p'], out_lm['biasSQ_lm'], 'o-', label='Bias$^2$')
ax.plot(out_lm['p'], out_lm['variance_lm'], 's-', label='Variance')
ax.plot(out_lm['p'], out_lm['MSE'], 'k^-', label='MSE')
ax.set(xlabel='p', ylabel='error', title='Linear regression'); ax.legend()
fig.tight_layout(); plt.show()
```







Discussion

Linear regression. The squared bias is almost constant, 0.028 for $p = 1$ to 4 and rising only to 0.054 at $p = 20$, because the linear fit cannot capture $\sin(2x_1)$ regardless of how many noise predictors are added. The variance grows almost linearly with p: 0.010, 0.015, 0.018, 0.026, 0.070 and 0.178 for $p = 1, 2, 3, 4, 10, 20$, close to the theoretical $\sigma^2(p+1)/n = 0.25\,(p+1)/50$, which gives 0.010 at $p = 1$ and 0.105 at $p = 20$. The total MSE therefore rises from 0.038 to 0.233, driven entirely by variance. The

decomposition $\mathrm{MSE} = \mathrm{Bias}^2 + \mathrm{Var}$ holds to four decimals in every row, as it should since the test targets are noise-free.

KNN, effect of K. For every p the variance decreases with K almost exactly as σ^2/K: about 0.245 at $K = 1$, 0.125 at $K = 2$ and 0.028 at $K = 9$, and it hardly depends on p at all. The squared bias moves the other way but much more weakly over $K = 1$ to 9, so within this range the MSE is minimised at large K (the optimal K is 8 or 9 for most p, and 6 for $p = 4$). The U-shape in $1/K$ is therefore visible only at its right-hand side, where $K = 1$ and $K = 2$ are clearly worst.

KNN, effect of p: the curse of dimensionality. The squared bias at the best K explodes as noise predictors are added: 0.002 for $p = 1$, 0.013 for $p = 2$, 0.05 for $p = 3$, 0.09 for $p = 4$, 0.31 for $p = 10$ and 0.37 for $p = 20$. With $p = 20$ the bias alone exceeds the variance of the training noise, because the "nearest" neighbours in 20 dimensions are essentially random points in x_1, so the local average is close to the global mean of f, which is zero. The best achievable KNN MSE rises from 0.032 at $p = 1$ to 0.384 at $p = 20$, a twelvefold increase, while the variance is unchanged.

Comparison. With $p = 1$ KNN with $K = 9$ (MSE 0.032) beats linear regression (0.038), because the sine is non-linear and one dimension is enough for neighbours to be genuinely close. At $p = 2$ the two are roughly tied (0.045 against 0.042). From $p = 3$ onward linear regression wins for every K, and at $p = 20$ its MSE of 0.23 is well below the best KNN value of 0.38. Linear regression pays for extra noise predictors with variance that grows linearly, whereas KNN pays with bias that grows much faster. This reproduces the pattern of FIGURE 3.20: a parametric model with a wrong but reasonable functional form is more robust to irrelevant predictors than a non-parametric one, and the advantage of flexibility disappears as soon as the effective dimension of the problem is larger than one or two.

In [31]:
```
# Export to HTML. Save the notebook first, then set EXPORT = True and run th
EXPORT = False
if EXPORT:
    !jupyter nbconvert --to html MATH_4432_Assignment_1.ipynb
```